

Topological Data Analysis

Lecture 15

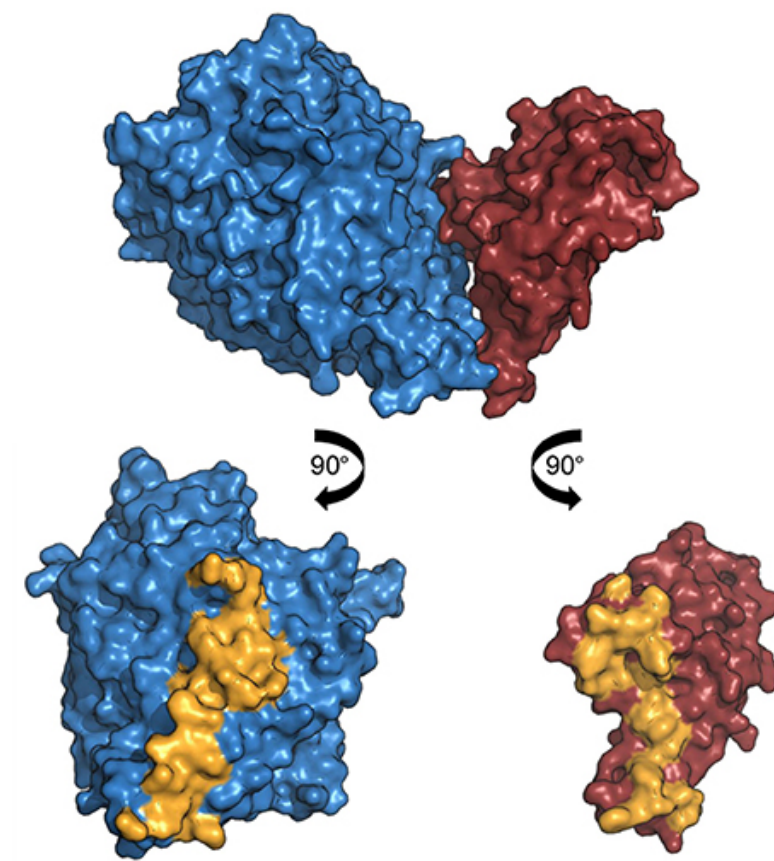
Topological Data Analysis of Graphs and Simplicial Complexes

Oleg Kachan

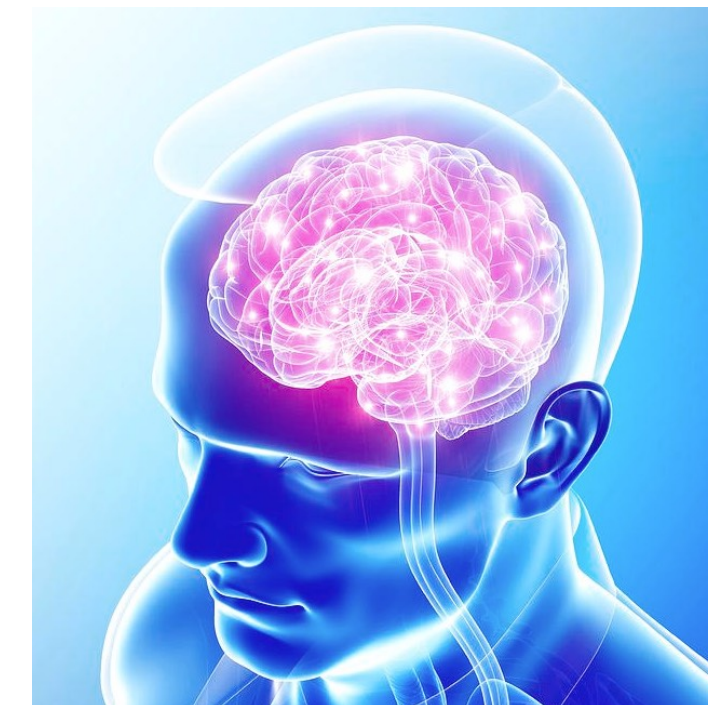
Complex networks



Social and collaboration networks



Protein interaction networks



Functional brain networks



Financial networks

Clique complex

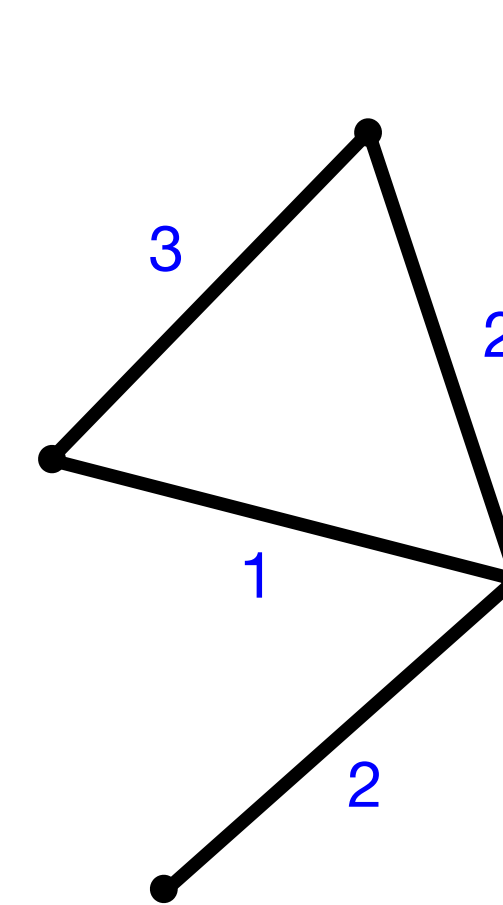
- An (abstract) simplicial complex K on a vertex set X is a family of subsets of X .
That is, if $\{x_0, \dots, x_p\} \subseteq K$, then any subset $\{x_{i_0}, \dots, x_{i_k}\} \subseteq K$.
- Given a graph $G = (V, E)$, its clique complex $K(G)$ is defined

$$K(G) = \begin{cases} K_0 &= V, \\ K_1 &= E, \\ K_p &= \left\{ \{x_0, \dots, x_p\} \mid \{x_i, x_j\} \in E, \forall i, j \right\}. \end{cases}$$

Weighted clique complex filtration

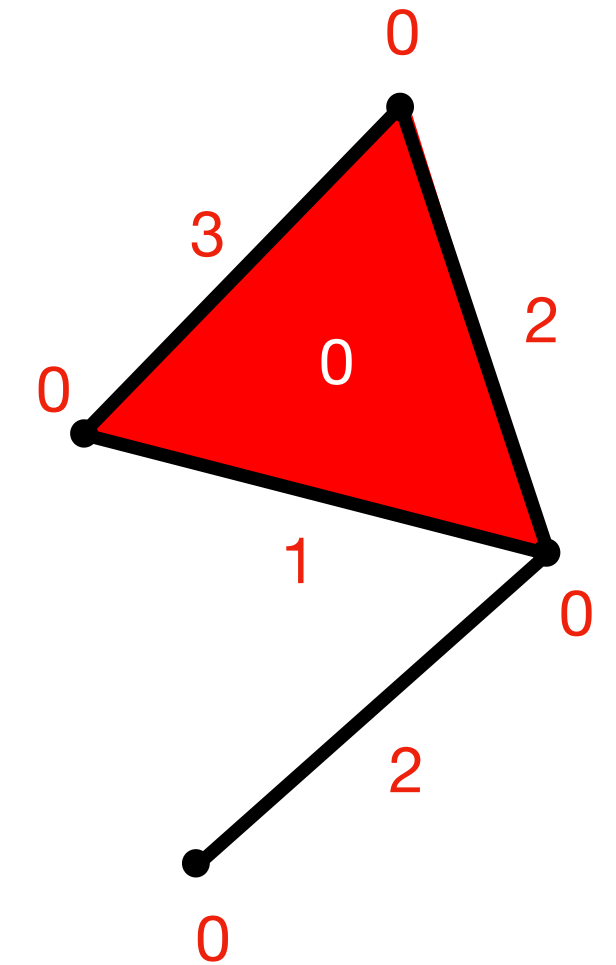
- Given a graph G with edge weights, defined by a function $w : E \rightarrow \mathbb{R}$
an induced weighted clique complex filtration function $f_w : K \rightarrow \mathbb{R}$ is defined

$$f_w(\sigma) = \begin{cases} 0, & \sigma \in K_0, \\ w(\sigma), & \sigma \in K_1, \\ \max_{e \in \sigma} w(e), & \sigma \in K_p, \quad p \geq 1. \end{cases}$$



Weighted graph G

$$w : E \rightarrow \mathbb{R}$$



Clique complex $K(G)$

$$f_w : K \rightarrow \mathbb{R}$$

Neighborhood complex

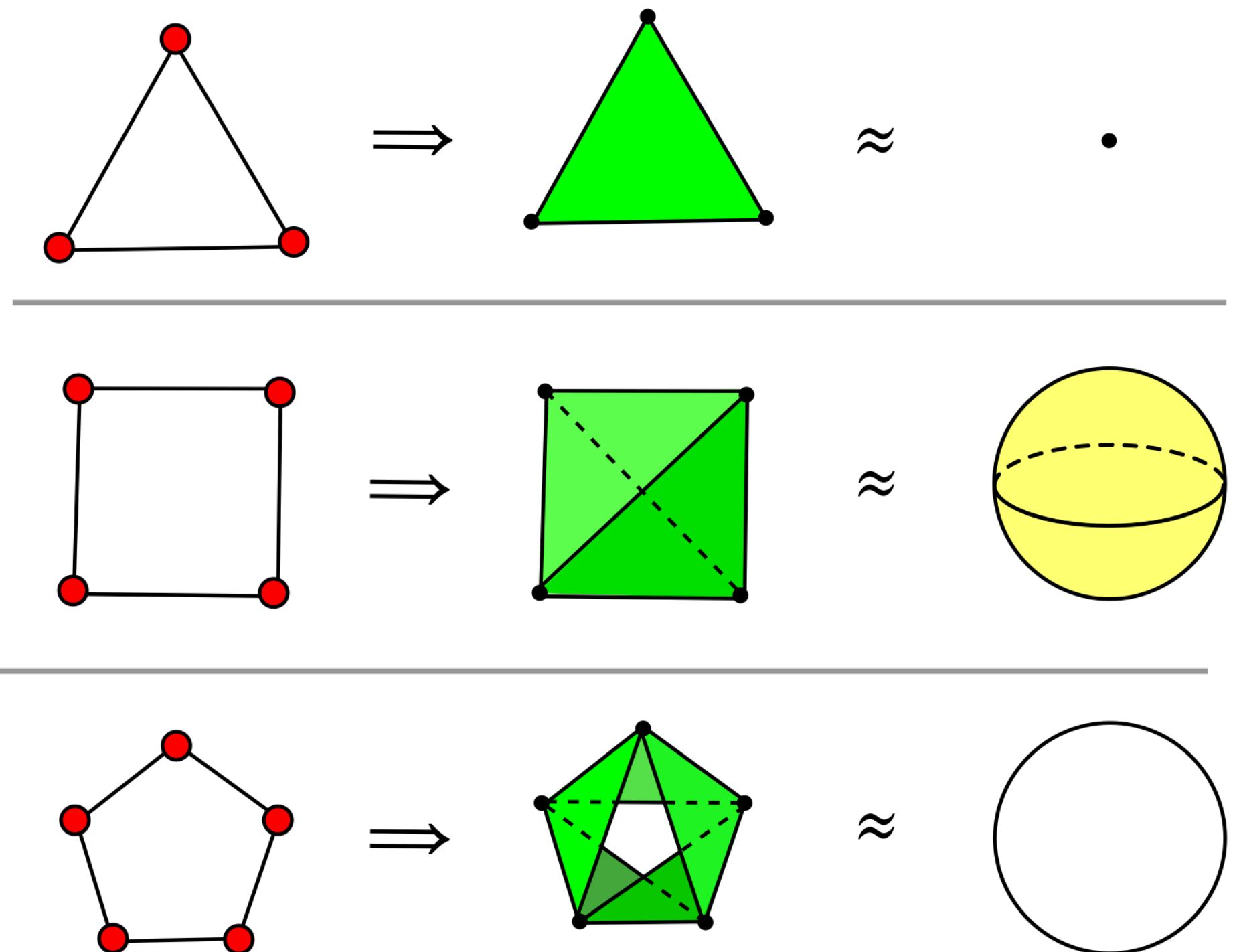
- Given a graph $G = (V, E)$, the neighborhood $N(v)$ of $v \in V$ is defined

$$N(v) = \{v\} \cup \{u \in V \mid \{v, u\} \in E\}$$

- Given a graph G , its neighborhood complex $K(G)$ is defined as the set of all nonempty subsets of neighborhoods of vertices of G . That is, if $N(v) \in K$, then all its subsets are also in K .

Weighted neighborhood complex filtration

- Given a graph G with edge weights, defined by a function $w : E \rightarrow \mathbb{R}$ an induced weighted neighborhood complex filtration function $f_w : K \rightarrow \mathbb{R}$
 - for every 0-simplex $\sigma = \langle v \rangle$, $f_{Nb}(\sigma)$ is the minimum value of f on the edges incident on v ;
 - calling G_t the subgraph induced by all edges e with $f(e) \leq t$, for a p -simplex σ , $p \geq 1$, we set $f(\sigma) = t$, where t is the smallest value of t for which σ is a subset of a neighborhood in G_t .



Neighborhood complexes

Neighborhood complex

Relation to the Dowker complex

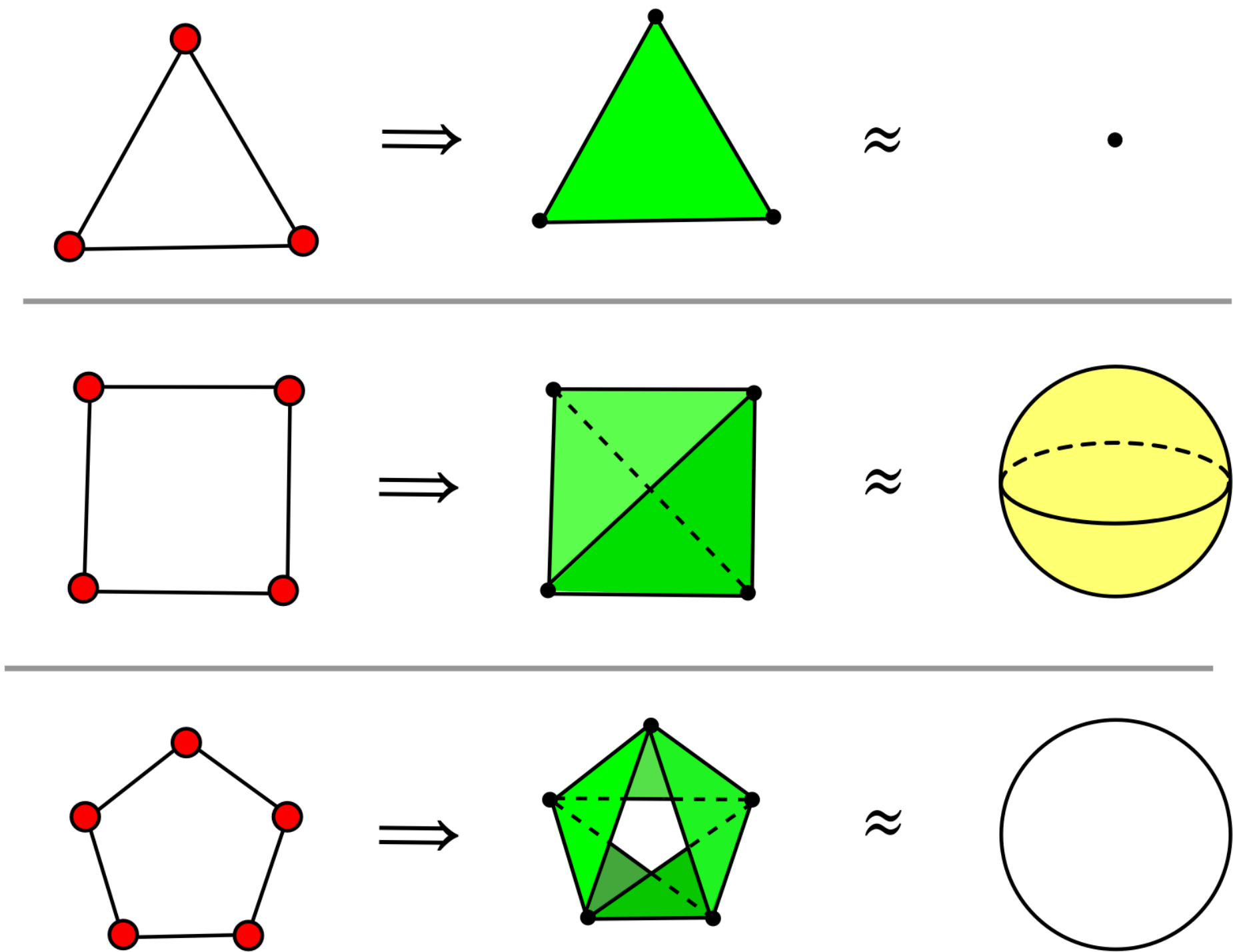
- Let R is a binary relation on set X , its Dowker complex K is defined

$$K = \{\sigma^{(p)} = \{x_{\sigma_0}, \dots, x_{\sigma_p}\} \mid \exists x \text{ s.t. } x_i R x \quad \forall x_i \in \sigma^{(p)}\}$$

A =

	x ₁	x ₂	x ₃	x ₄	x ₅
x ₁	1	1			1
x ₂	1	1	1		
x ₃		1	1	1	
x ₄			1	1	1
x ₅	1			1	1

$$N(v) = \{v\} \cup \{u \in V \mid \{v, u\} \in E\}$$



Neighborhood complexes

Simplicial complexes of graphs

- complex of independence sets of G ;
- complex of complementary sets to dominating sets of G ;
- the nonempty sets $\sigma \subseteq V$ such that the subgraph induced by σ is acyclic;
- the nonempty sets $\sigma \subseteq E$ such that the subgraph induced by σ is acyclic;
- (with G connected) the nonempty sets $\sigma \subseteq E$ such that the subgraph induced by σ is acyclic and the subgraph induced by $E - \sigma$ is connected;
- for fixed positive $j < k$, the sets of k -cycles which share at least j vertices;
- for fixed positive l , the sets of maximal cliques sharing at least l vertices;
- etc.

Directed clique complex

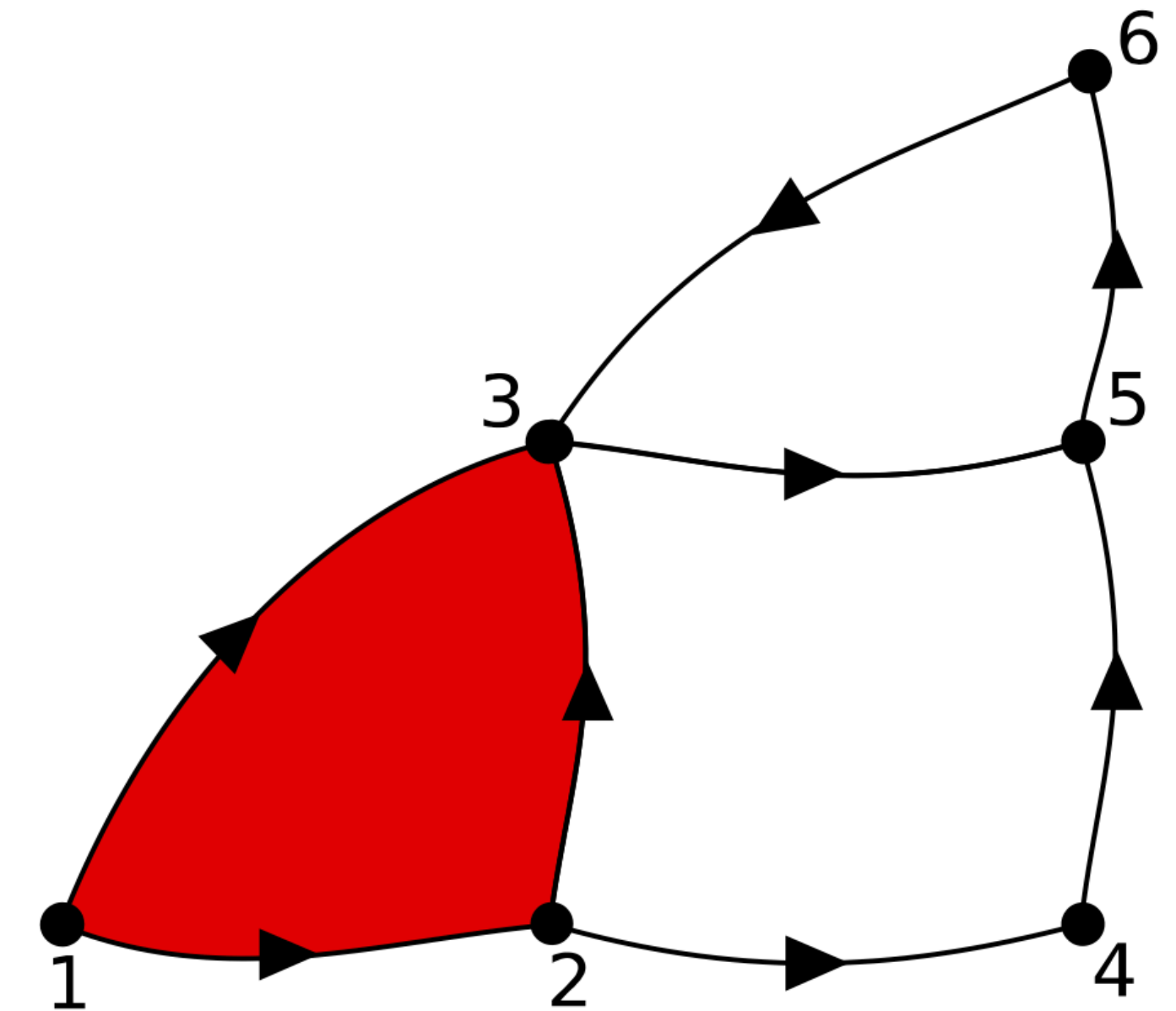
- An oriented simplicial complex K on a vertex set X is a family of ordered subsets of X . That is, if $(x_0, \dots, x_p) \subseteq K$, then any ordered subset $(x_{i_0}, \dots, x_{i_k}) \subseteq K$, $\forall 0 \leq k \leq p$.
- Given a directed graph $G = (V, E)$, its directed clique complex $K(G)$ is defined

$$K(G) = \begin{cases} K_0 &= V, \\ K_1 &= E, \\ K_p &= \{(x_0, \dots, x_p) \mid (x_i, x_j) \in E, \forall i < j\}. \end{cases}$$

Weighted directed clique complex filtration

- Given a directed graph G with edge weights, defined by a function $w : E \rightarrow \mathbb{R}$ an induced weighted clique complex filtration function $f_w : K \rightarrow \mathbb{R}$ is defined

$$f_w(\sigma) = \begin{cases} 0, & \sigma \in K_0, \\ w(\sigma), & \sigma \in K_1, \\ \max_{e \in \sigma} w(e), & \sigma \in K_p, \ p \geq 1. \end{cases}$$

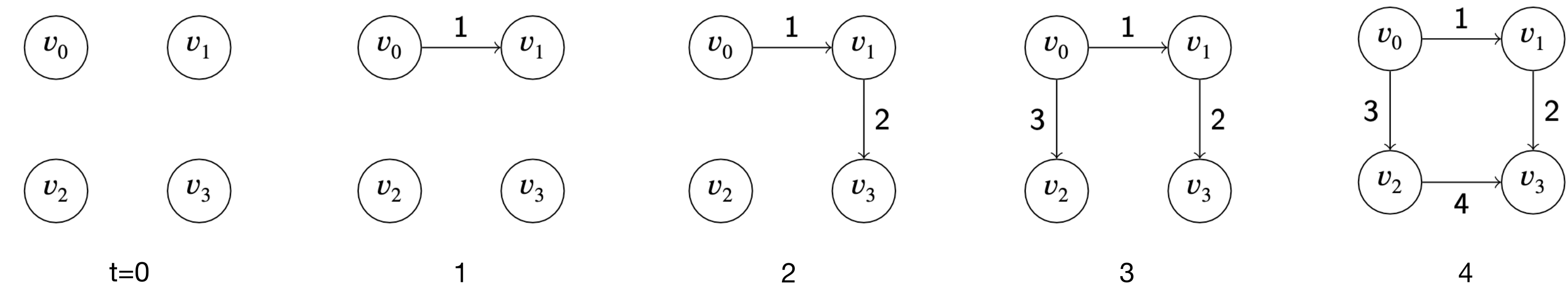


Directed clique complex

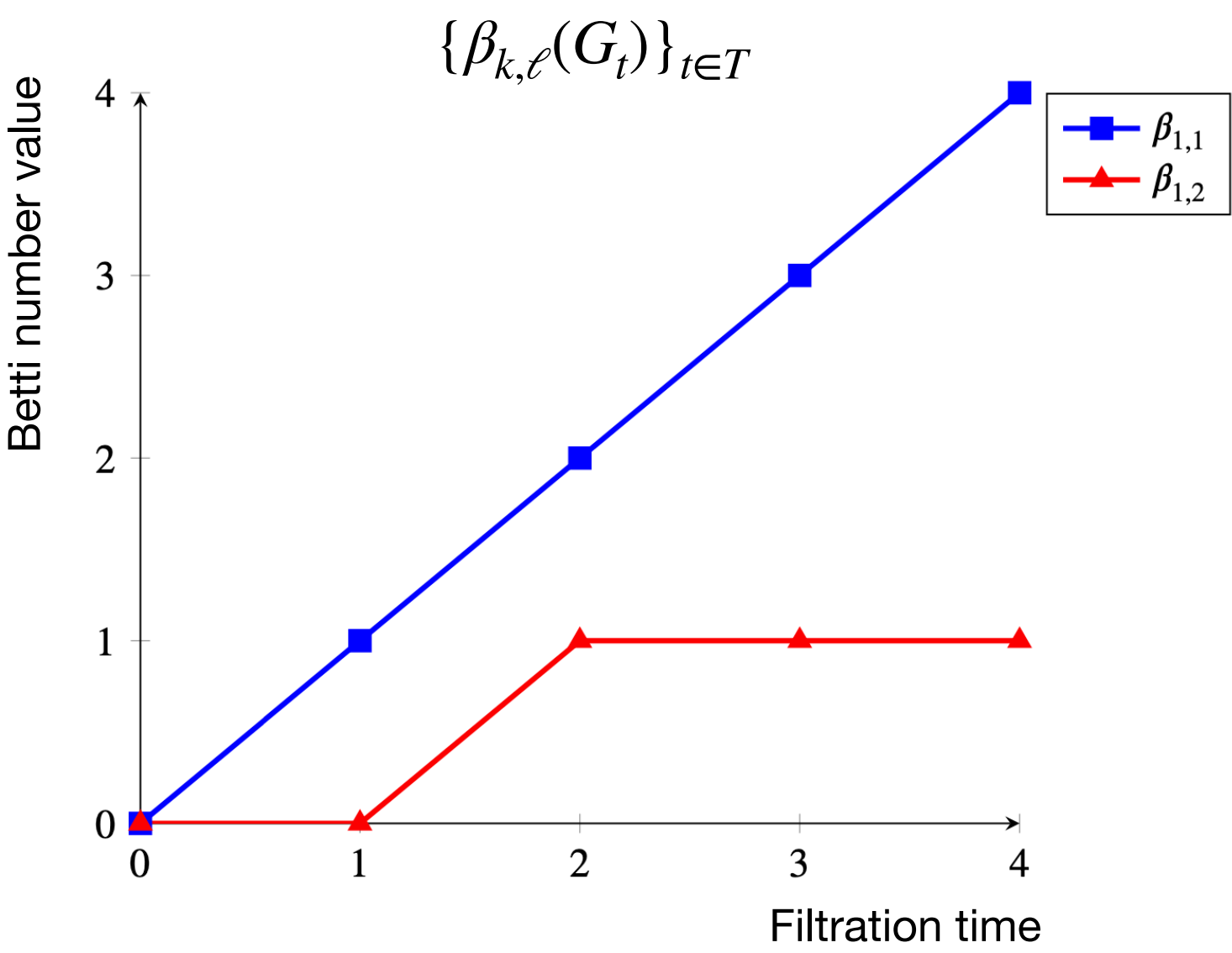
Magnitude homology

- Consider a directed graph $G = (V, E, s, t)$, define $d(u, v)$ as the length of a shortest path from a vertex u to v and the length of a tuple $\ell(v_0, \dots, v_k) = d(v_0, v_1) + \dots + d(v_{k-1}, v_k)$
- the (k, l) -Betti number $\beta_{k,\ell}$ of magnitude homology of G is the number of $(k+1)$ -tuples of vertices of length ℓ
- (k,l) -Betti curve $\beta_{k,\ell}(t)$ of G for a filtration $F = \{G_t\}_{t \in T}$ is a sequence $\{\beta_{k,\ell}(G_t)\}_{t \in T}$
- the $(1, l)$ -Betti number $\beta_{1,\ell}$ of magnitude homology of G is the number of pairs of vertices (u, v) with shortest paths $d(u, v) = \ell$

Filtration



Magnitude-Betti curve



Hochschild homology

- Let $G = (V, E, s, t)$ be a directed graph, where $s((v_0, v_1)) = v_0$, $t((v_0, v_1)) = v_1$ for $e \in E$
- *path* $\gamma = (e_1, \dots, e_n)$ of length n is a sequence of edges of G s.t. $t(e_i) = s(e_{i+1})$
- *path algebra* A of G is a vector space generated by all possible paths in G , with multiplication defined as

$$\gamma\gamma' = \begin{cases} (e_1, \dots, e_n, e'_1, \dots, e'_p), & s(e'_1) = t(e_n) \\ 0, & \text{otherwise} \end{cases}$$

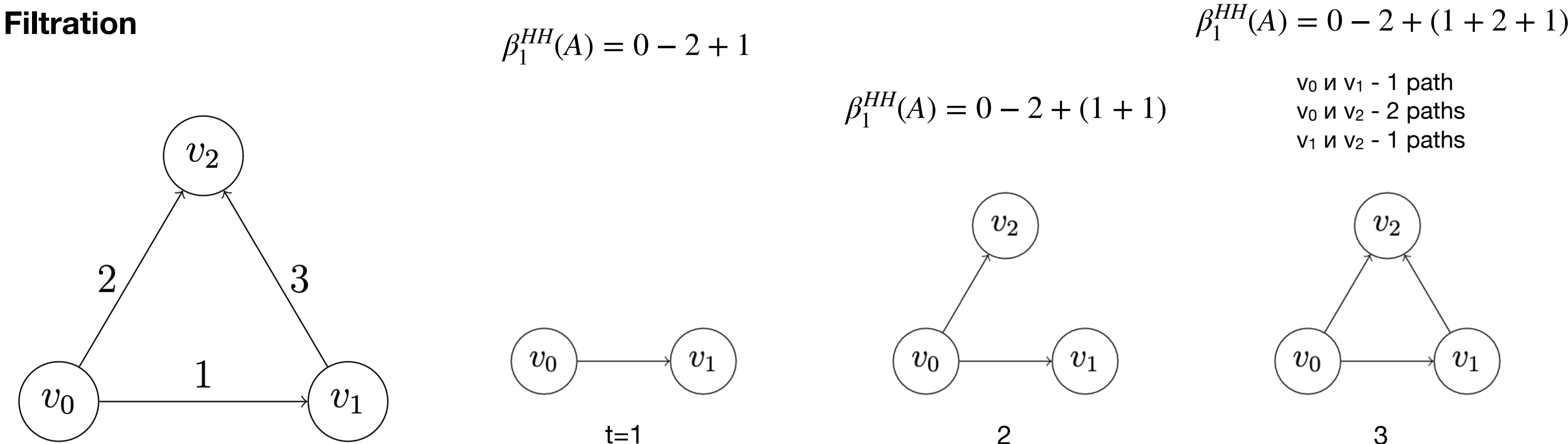
Hochschild-Betti numbers

Interpretable as

$$\beta_0^{HH}(A) = \#\text{weakly cc}(G)$$

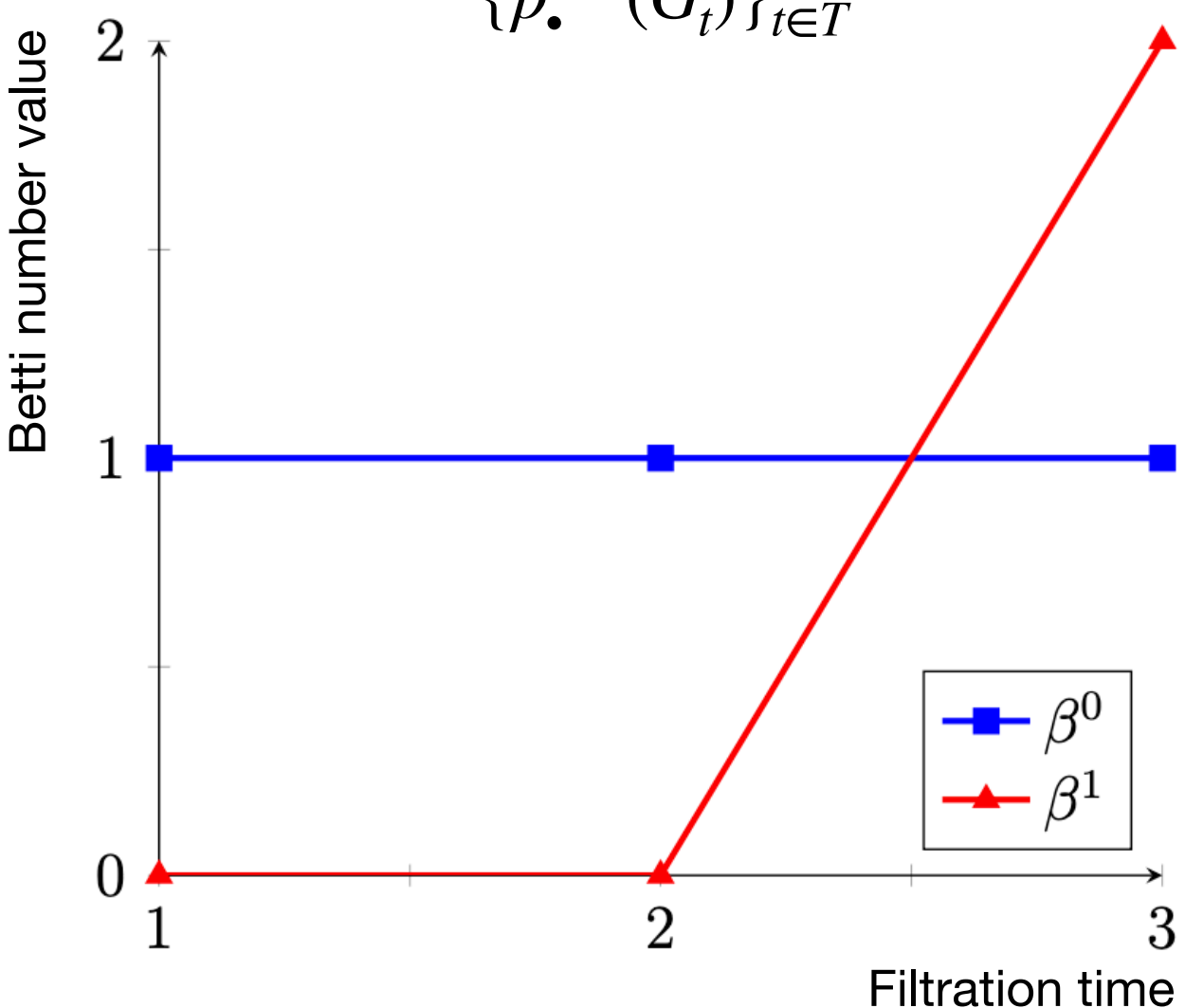
$$\beta_1^{HH}(A) = c - n + \sum_{e \in E(G)} \dim e_{t(e)} A e_{s(e)}$$

Filtration

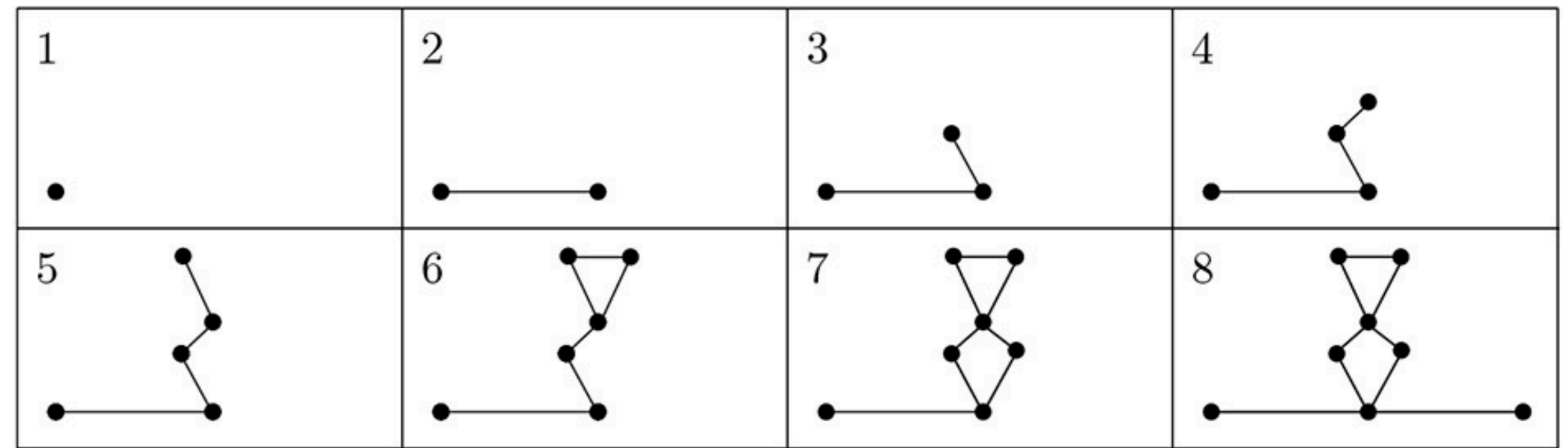
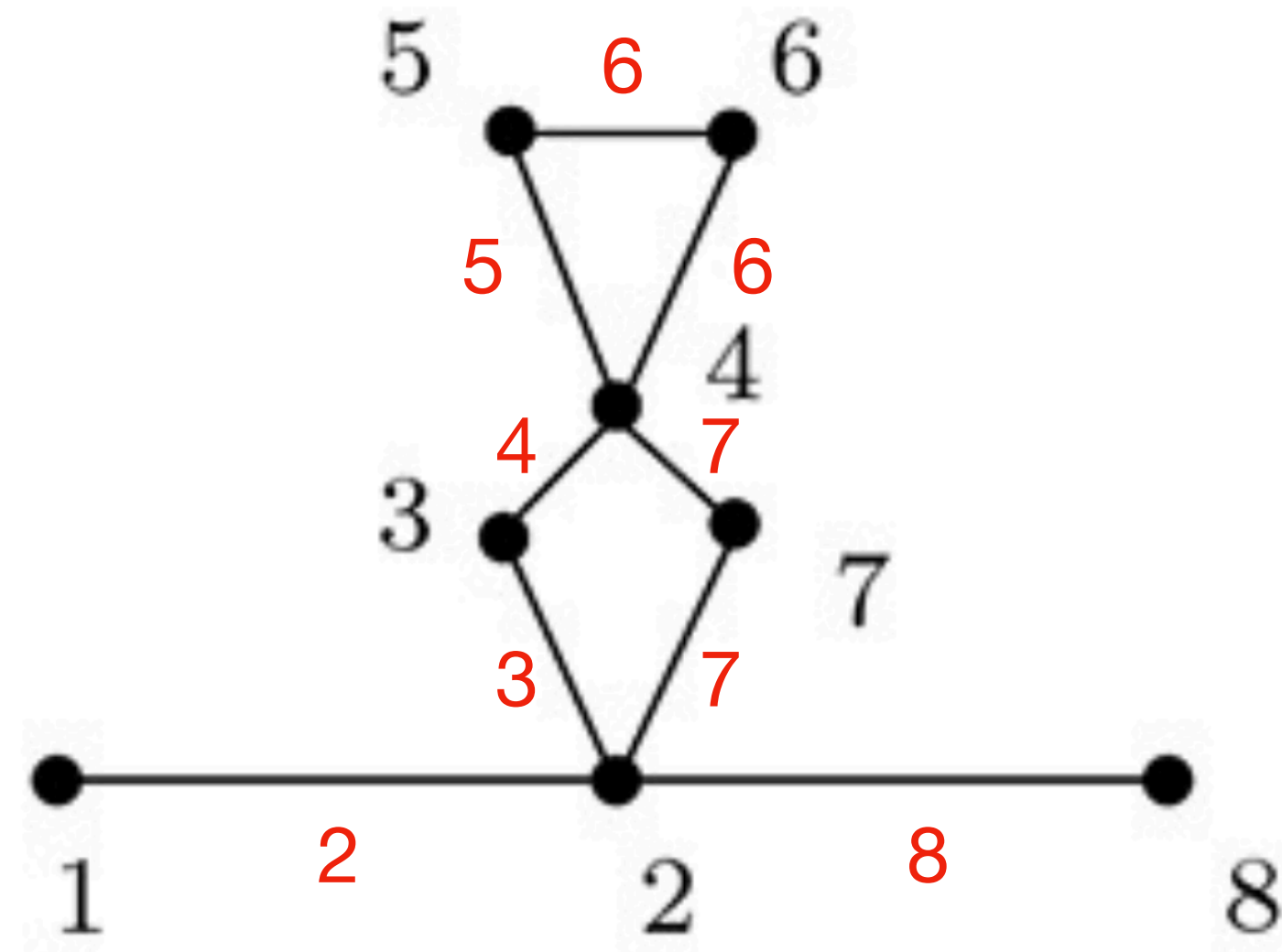


Hochschild-Betti curve

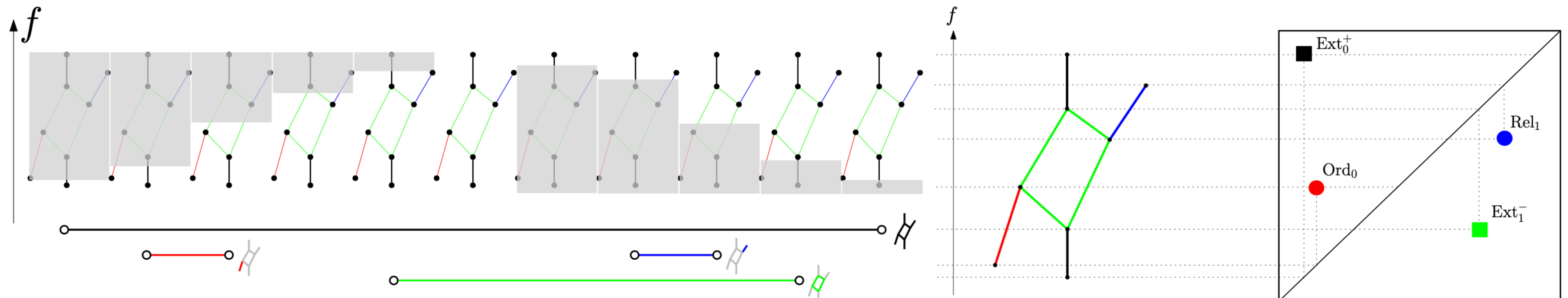
$$\{\beta_{\bullet}^{HH}(G_t)\}_{t \in T}$$



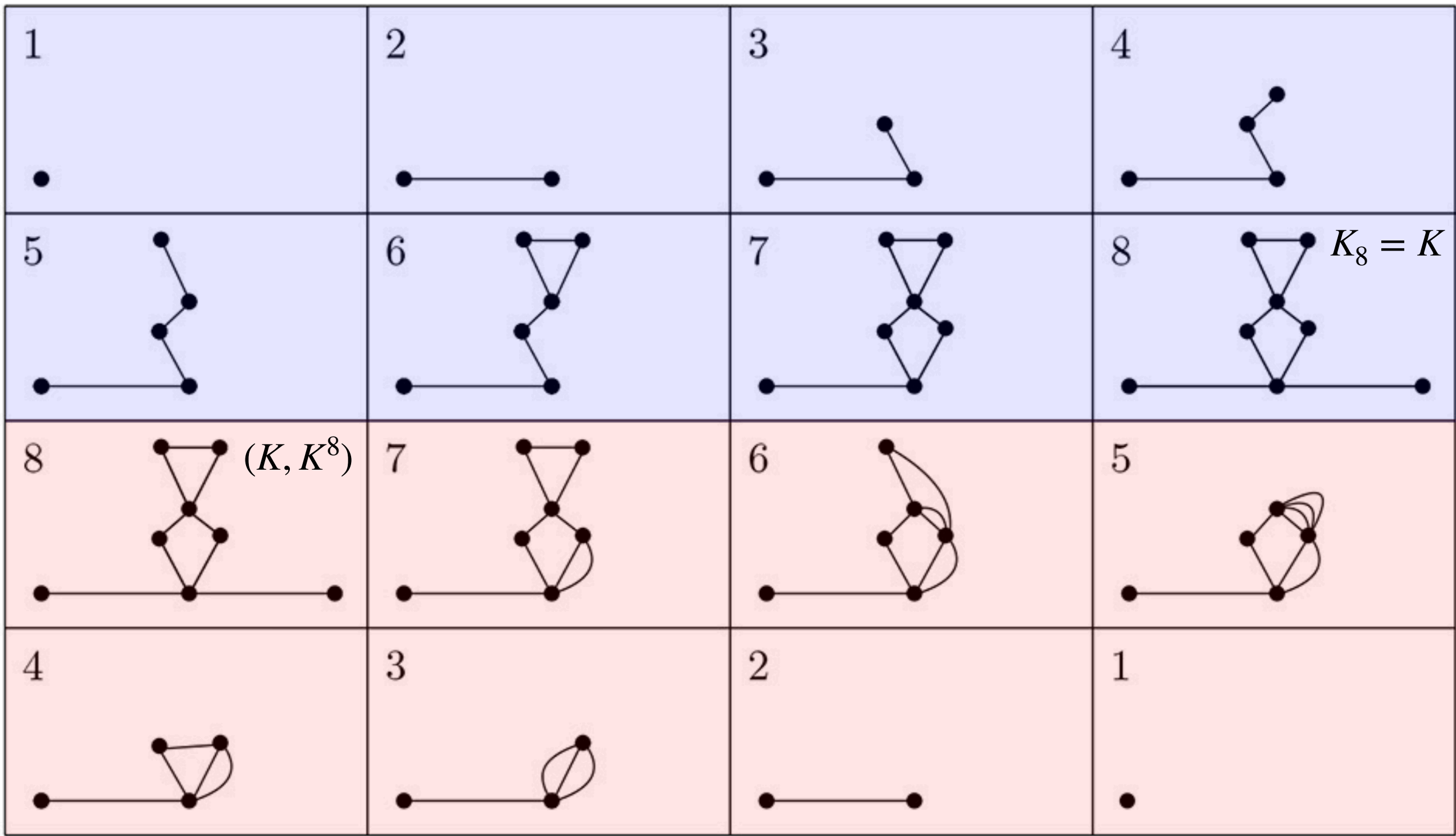
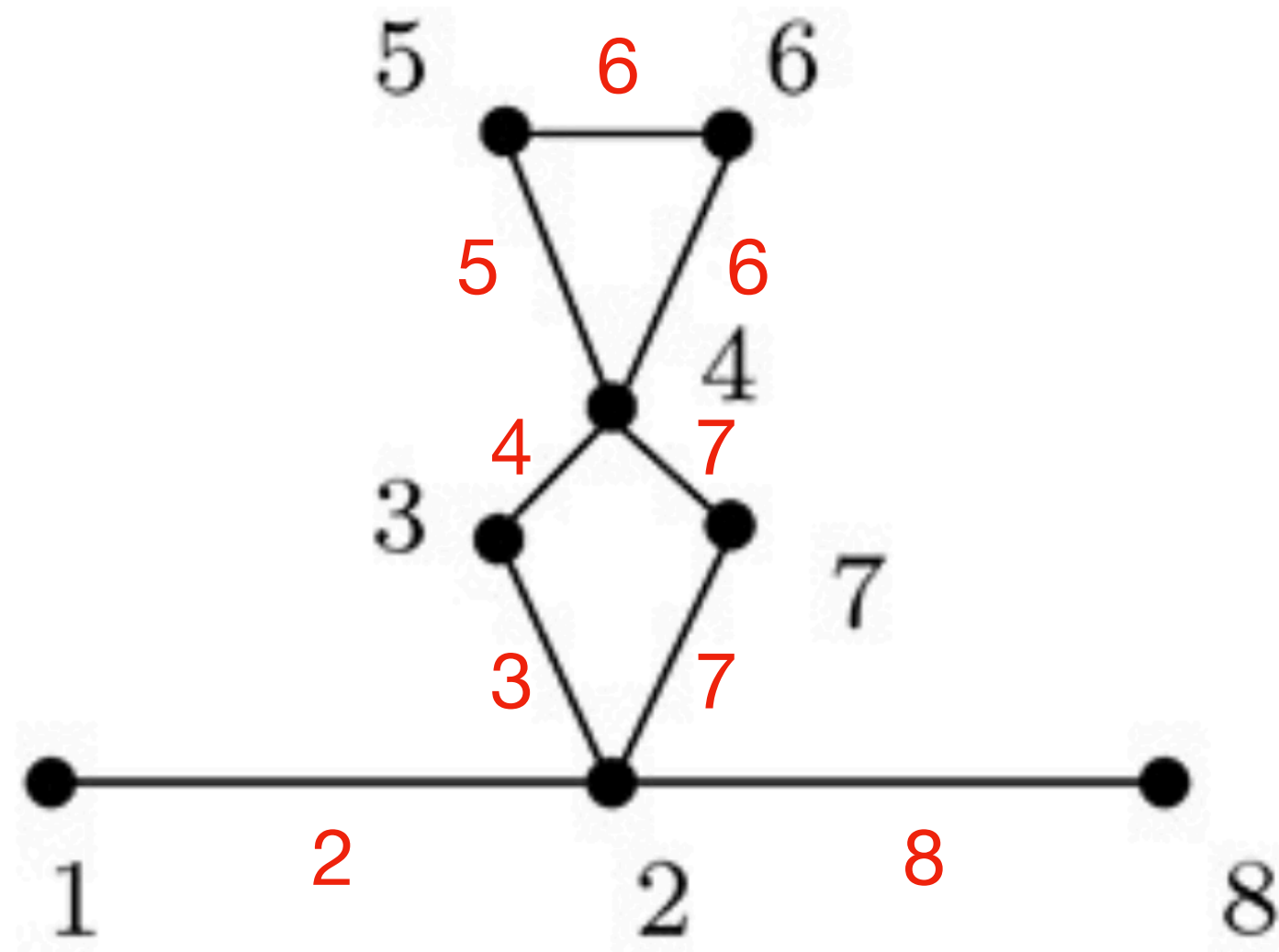
Extended persistence



- Loops persist forever, do not die:
 - loop 2347 begins to form at 2 and is completed at 7
 - loop 456 begins to form at 4 and is completed at 6
- “Upwards” branches w.r.t. the filtration function do not form connected components, when using the sublevel set filtration



Extended persistence



$$\emptyset = K_{-\infty} \subset K_1 \subset K_2 \subset \dots \subset K = (K, K^{\infty}) \subset \dots \subset (K, K^2) \subset (K, K^1) \subset (K, K^{-\infty}) = \emptyset$$



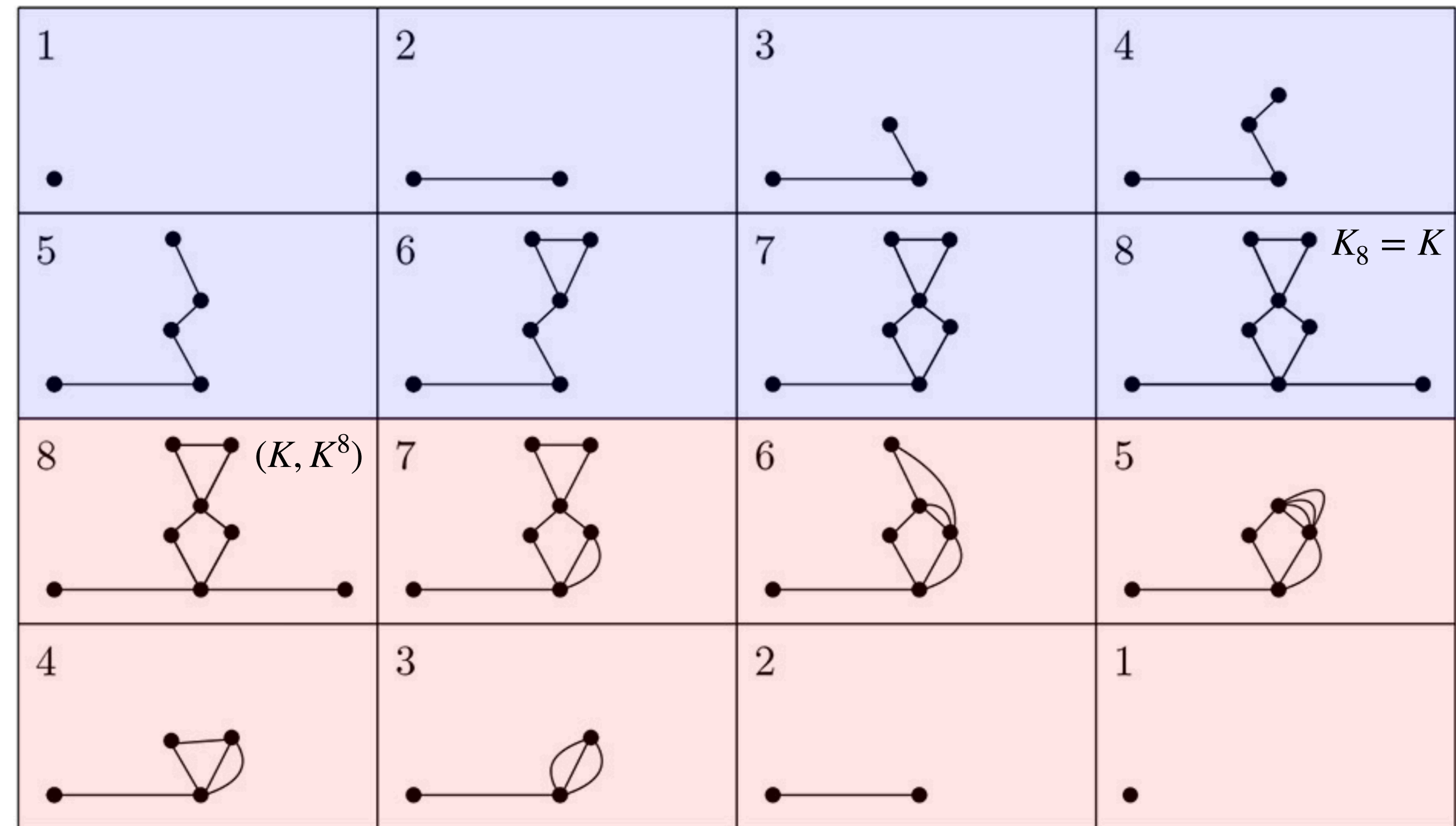
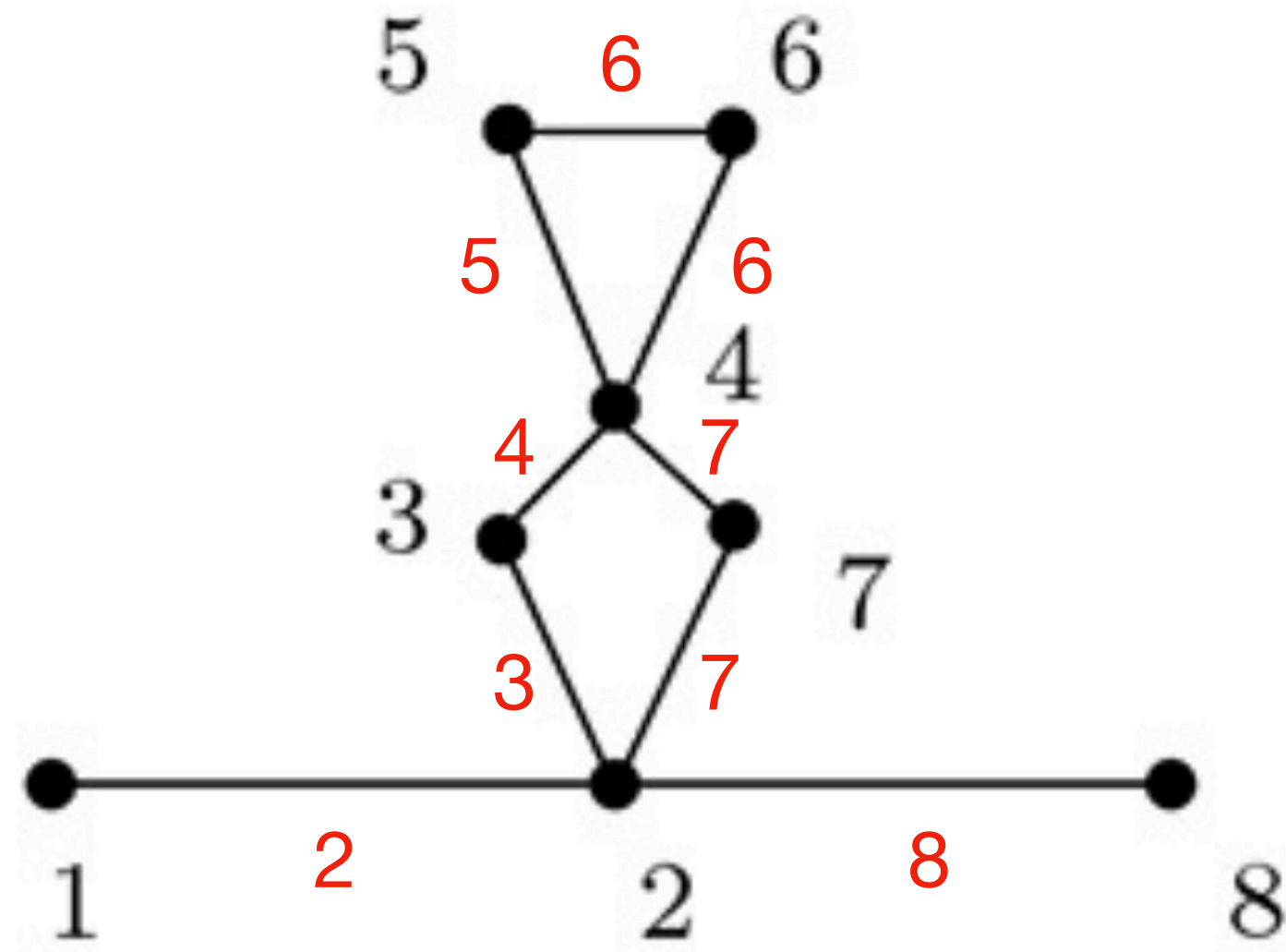
$$K_t = \{\sigma \in K \mid f(\sigma) \leq t\}$$

sublevel set $t \in (-\infty, +\infty)$

$$K^t = \{\sigma \in K \mid f(\sigma) \geq t\}$$

superlevel set $t \in (+\infty, -\infty)$

Extended persistence



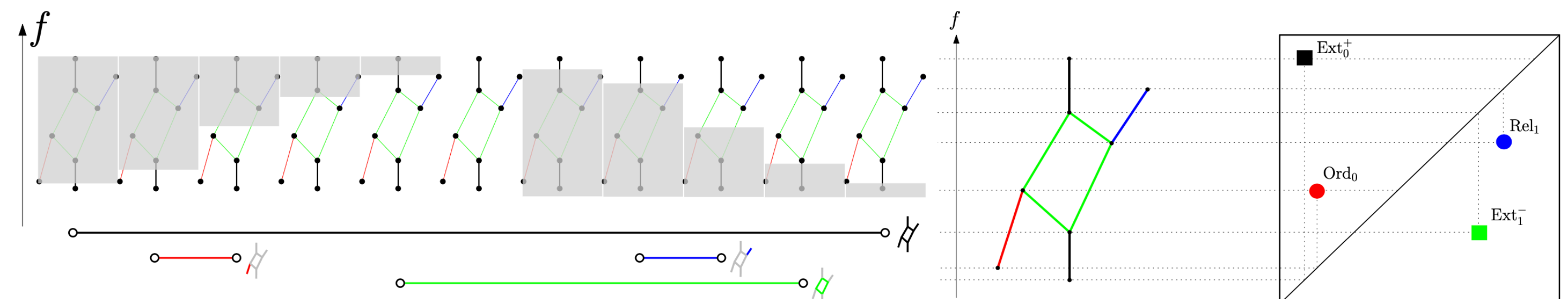
$$\emptyset = K_{-\infty} \subset K_1 \subset K_2 \subset \dots \subset K = (K, K^{\infty}) \subset \dots \subset (K, K^2) \subset (K, K^1) \subset (K, K^{-\infty}) = \emptyset$$



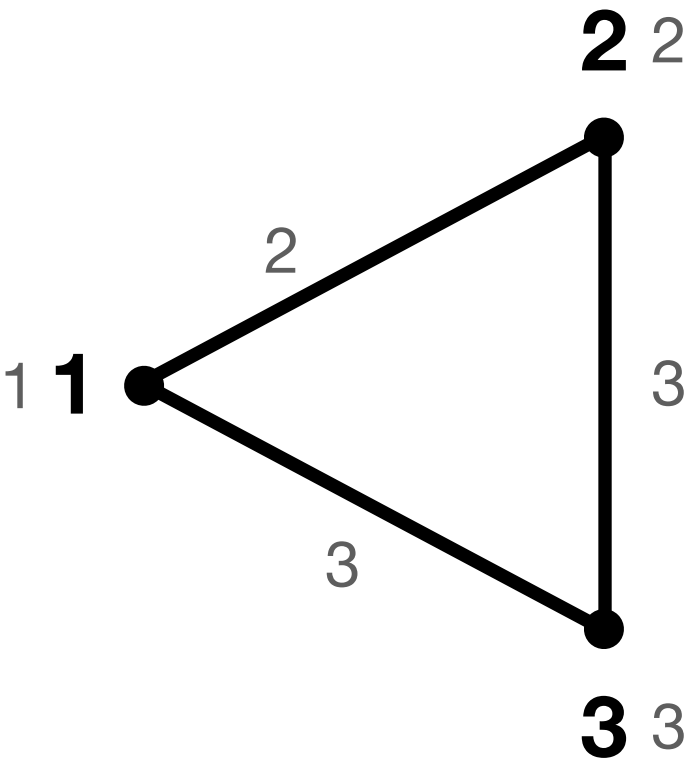
filtration of sublevel sets of f

filtration relative to superlevel sets of f

- **ordinary** — born and die in the ordinary part
- **relative** — born and die in the relative part
- **extended** — born and die in different parts

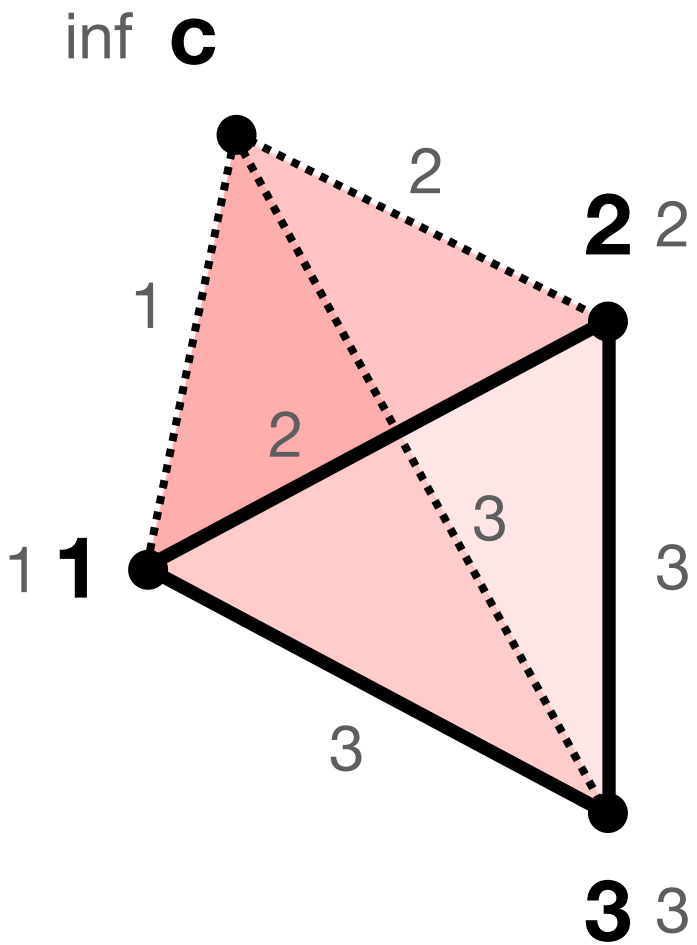


Extended persistence



		1	2	2	3	3	3
		1	2	12	3	13	23
1	1			1		1	
2	2			1			
2	12						
3	3					1	
3	13						
3	23						

Extended persistence



		1	2	2	3	3	3	inf	3	2	2	1	1	1
		1	2	12	3	13	23	c	c3	c2	c23	c1	c12	c13
1	1			1		1						1		
2	2			1						1				
2	12												1	
3	3					1			1					
3	13													1
3	23										1			
inf	c								1	1		1		
3	c3										1			1
2	c2										1		1	
2	c23													
1	c1												1	1
1	c12													
1	c13													

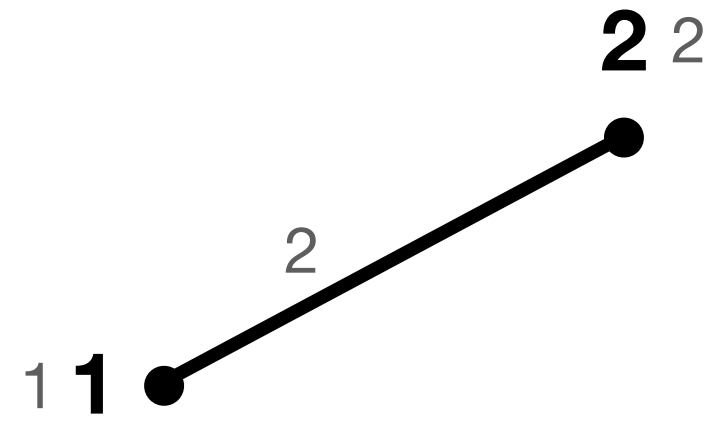
11 ●

[illegible]

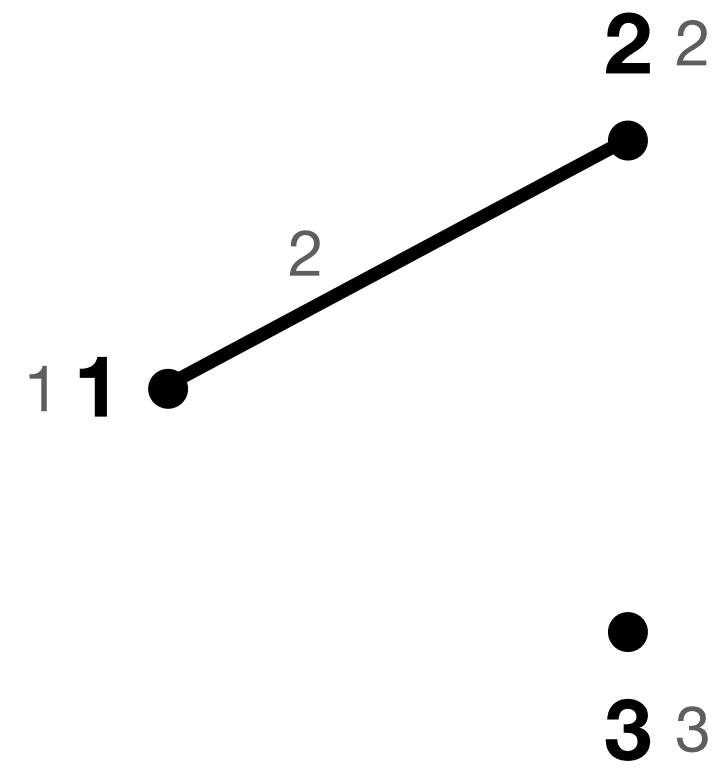
Extended persistence

[illegible]

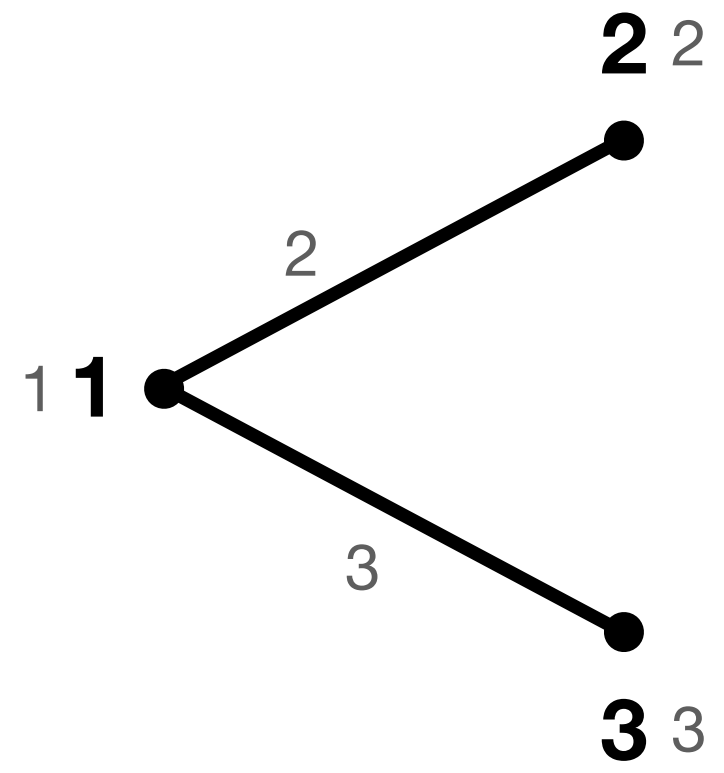
Extended persistence

[illegible]

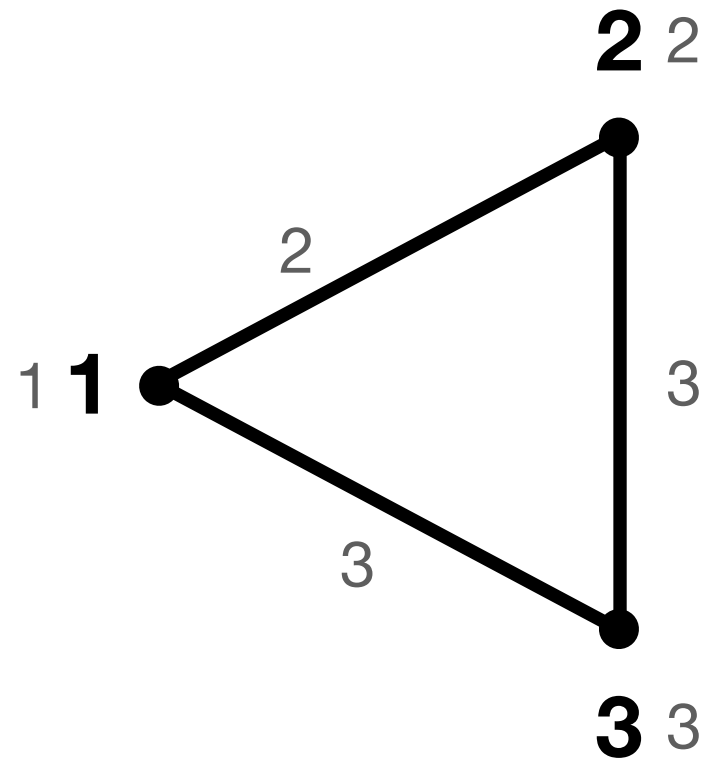
Extended persistence

[illegible]

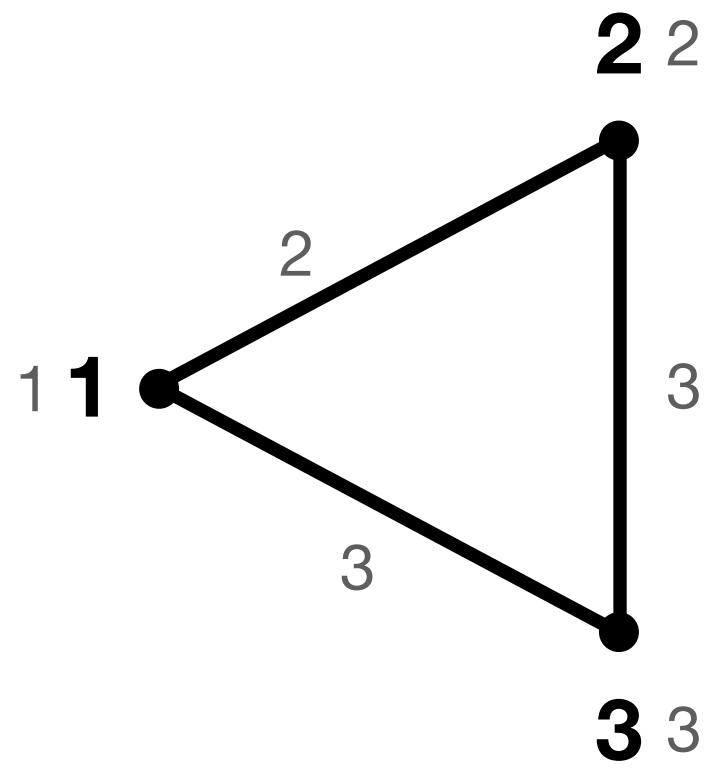
Extended persistence

[illegible]

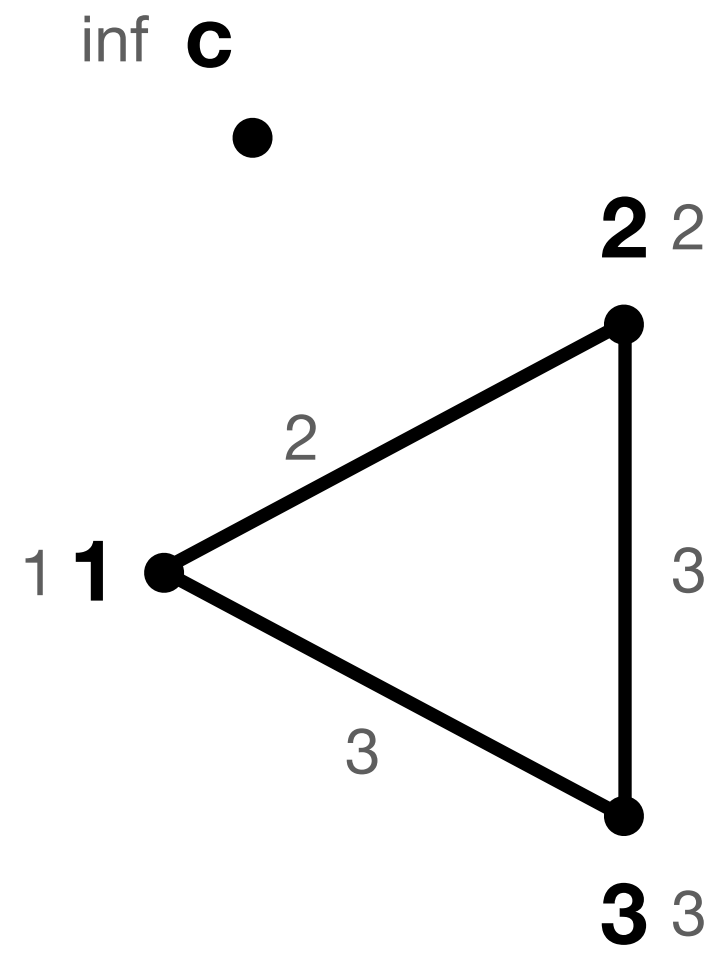
Extended persistence

[illegible]

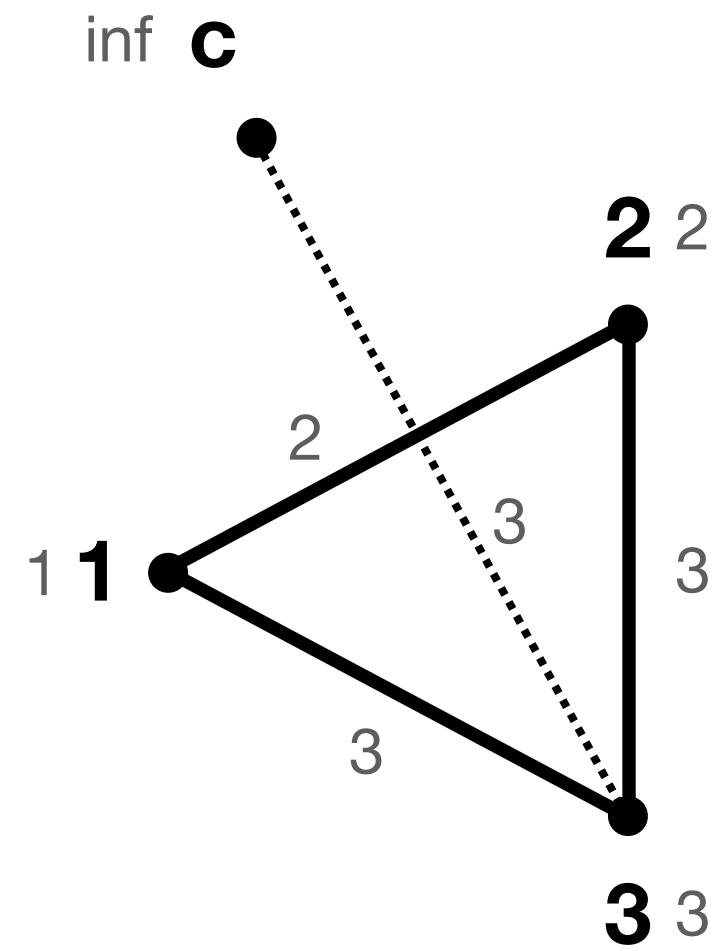
Extended persistence

[illegible]

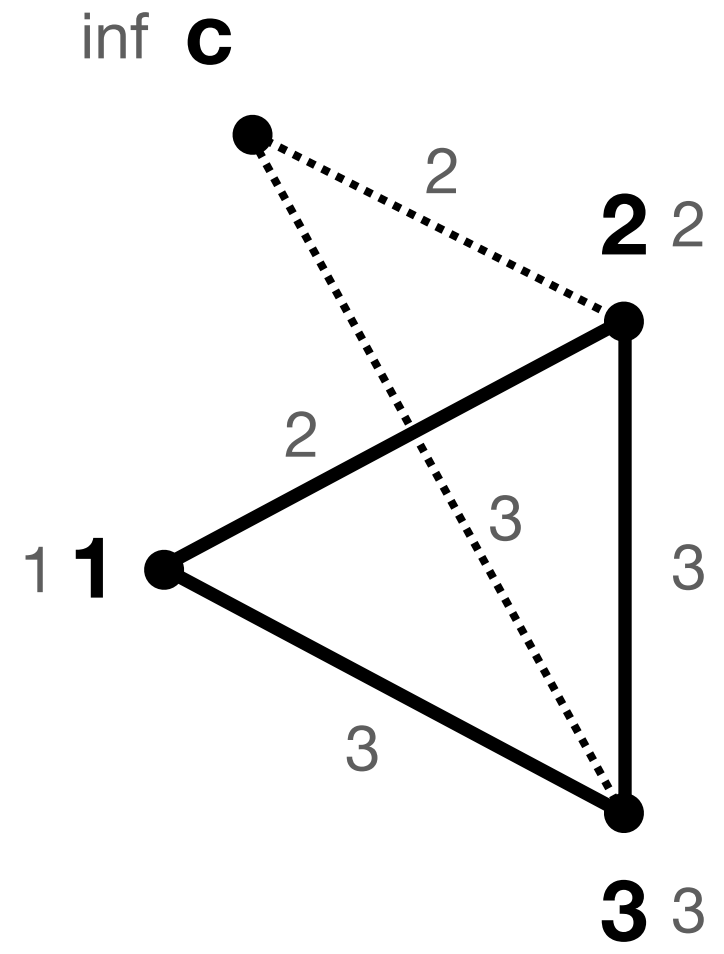
Extended persistence

[illegible]

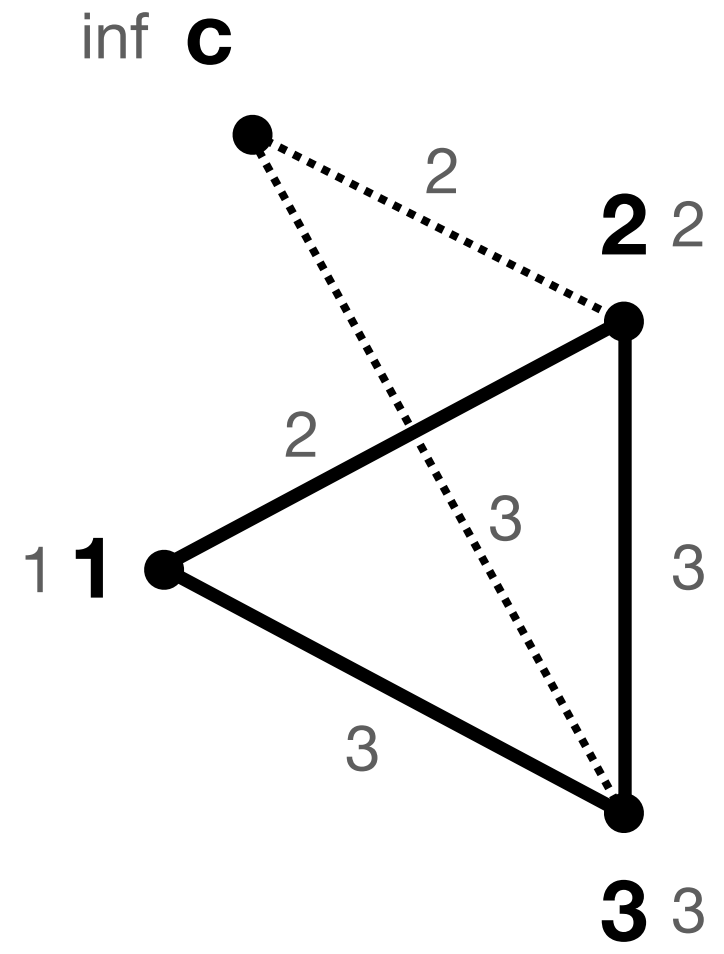
Extended persistence

[illegible]

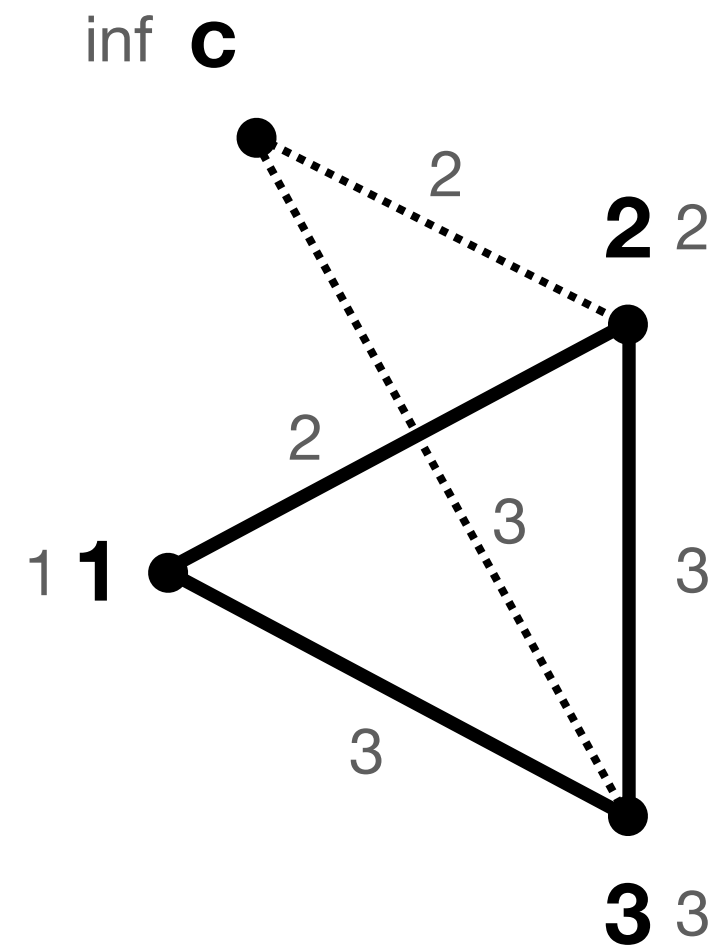
Extended persistence

[illegible]

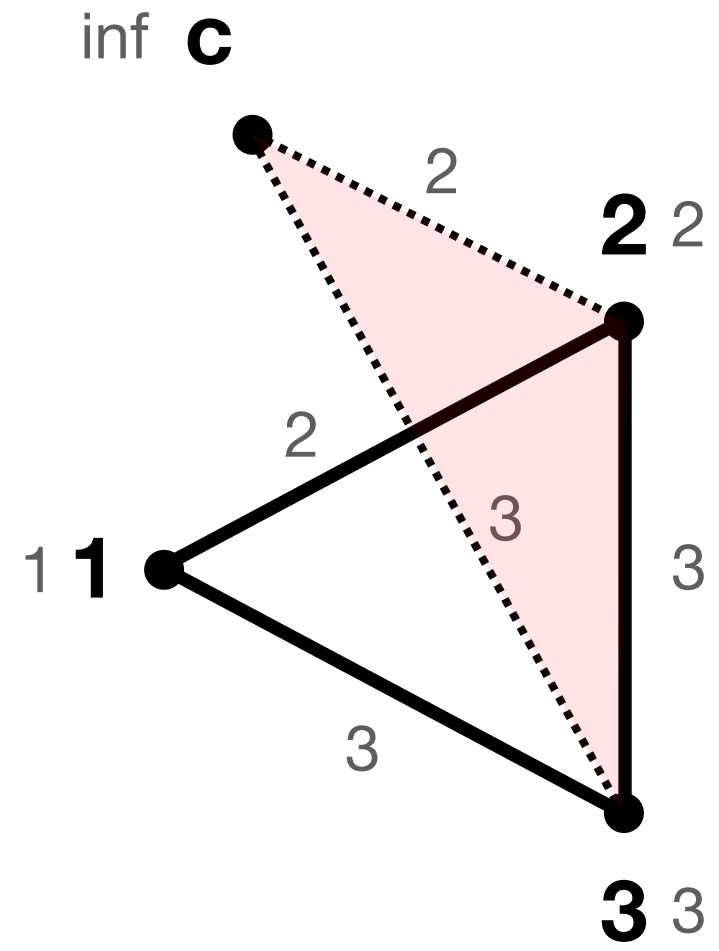
Extended persistence

[illegible]

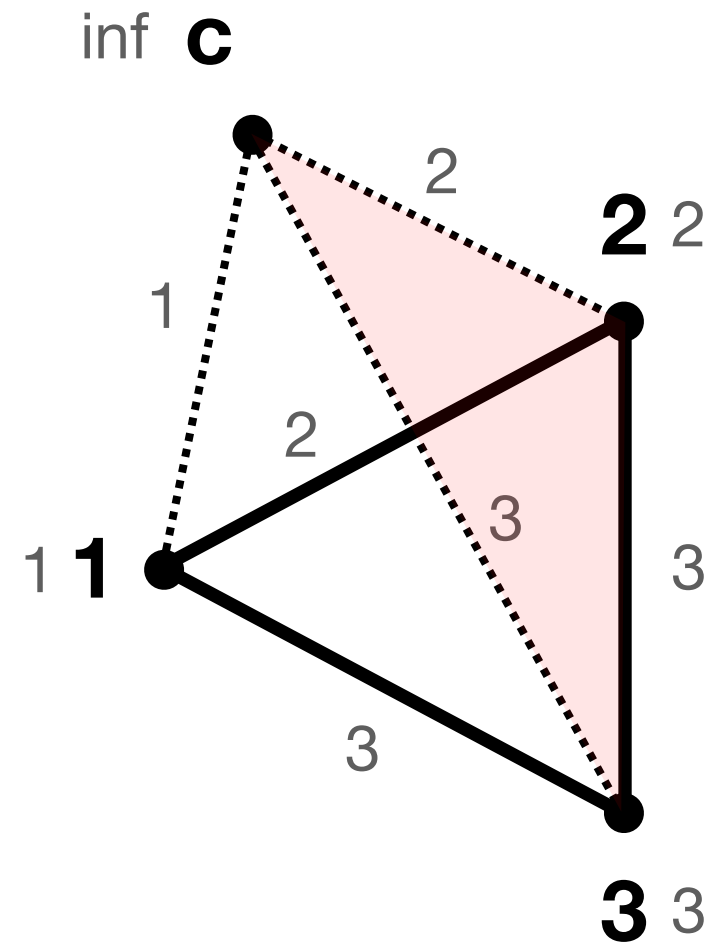
Extended persistence

[illegible]

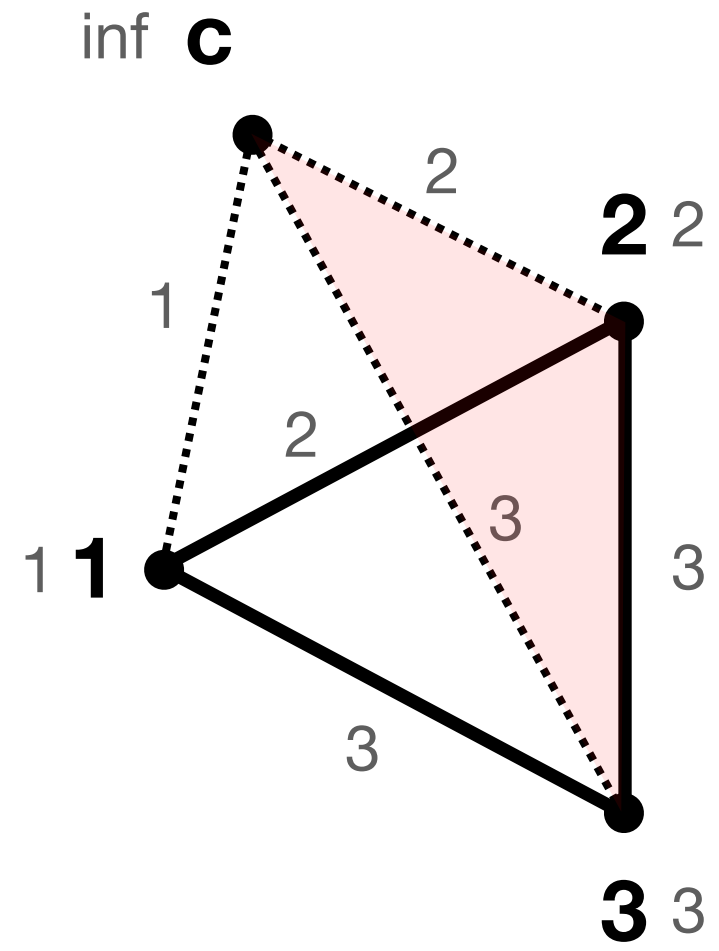
Extended persistence

[illegible]

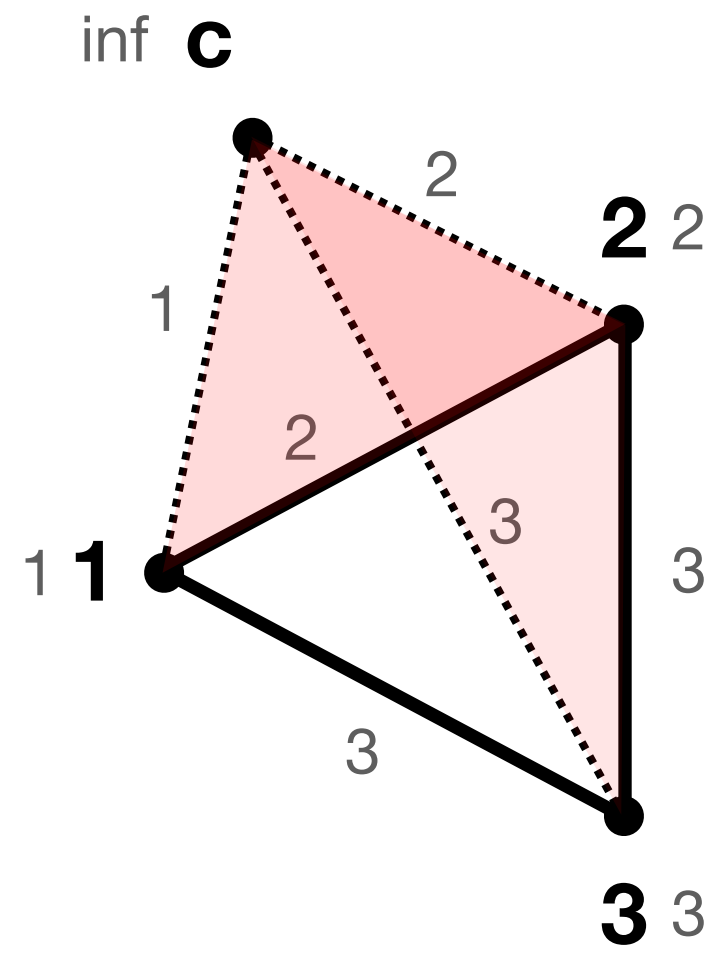
Extended persistence

[illegible]

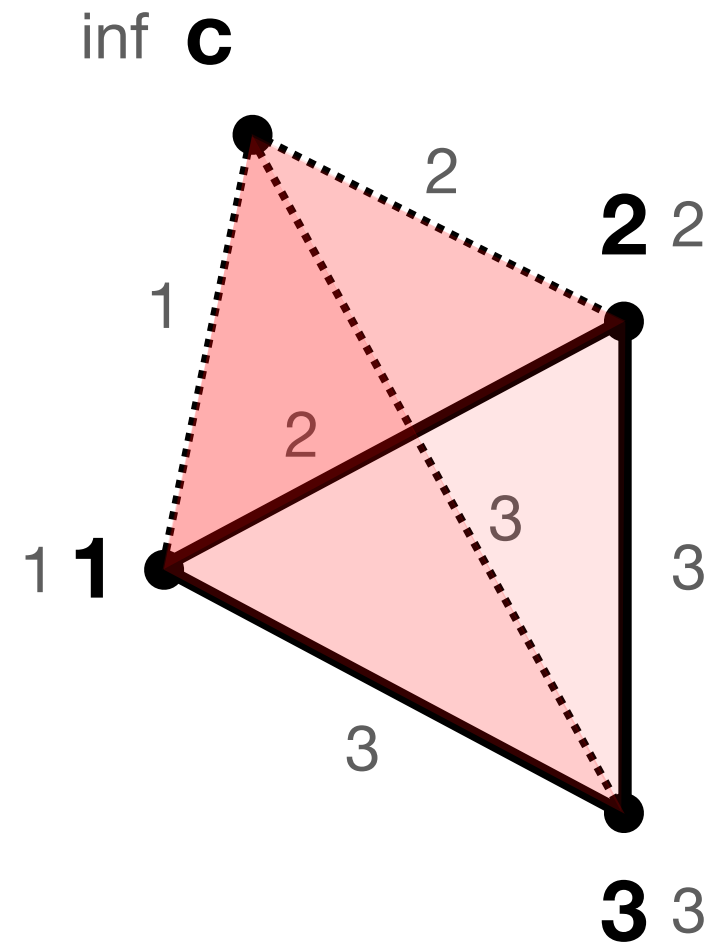
Extended persistence

[illegible]

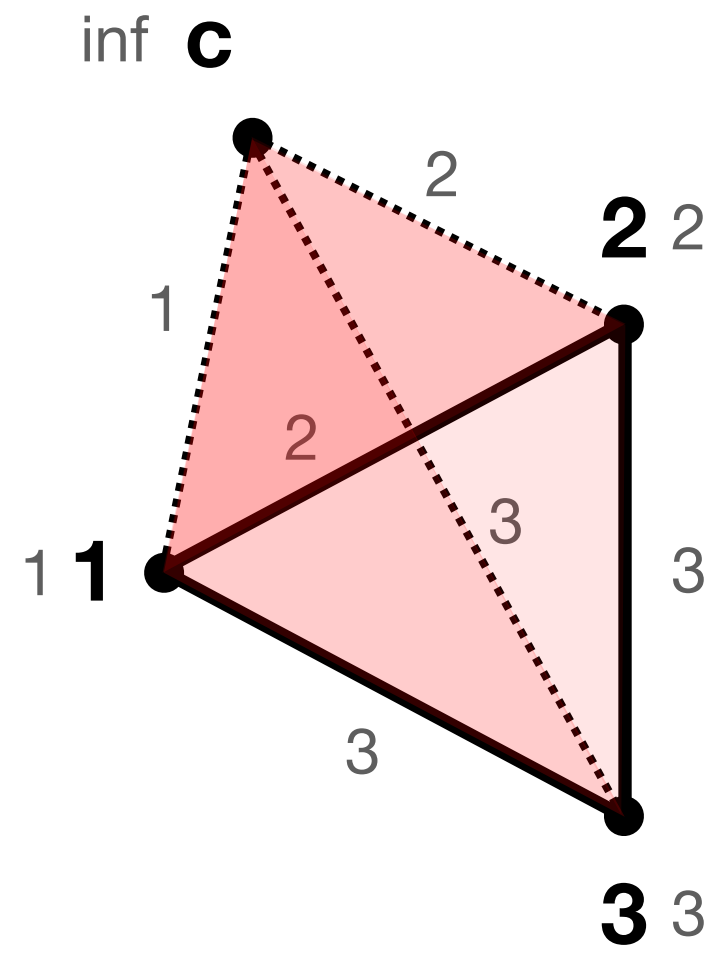
Extended persistence

[illegible]

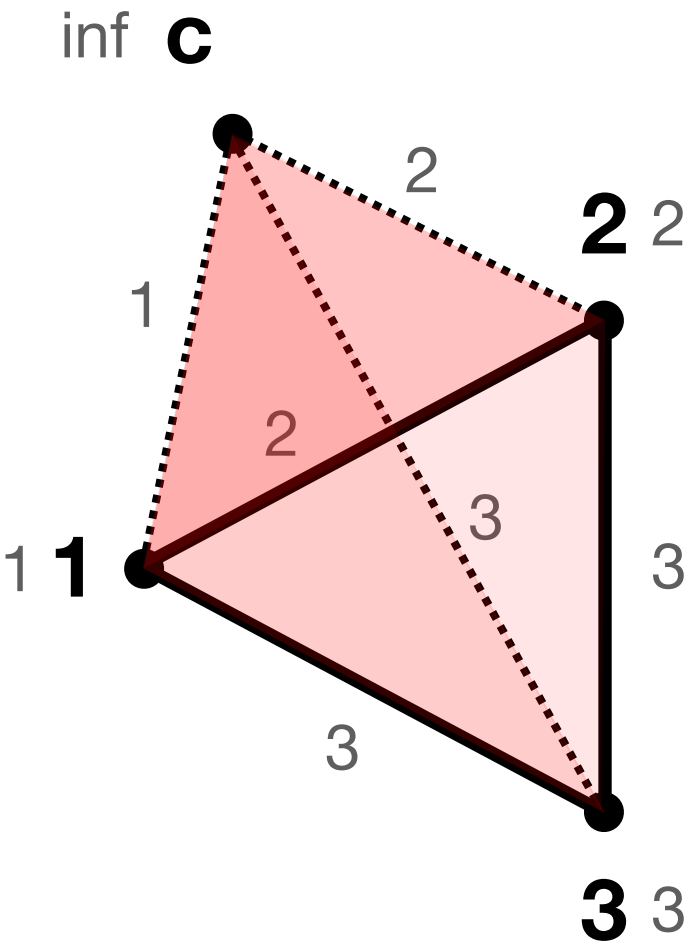
Extended persistence

[illegible]

Extended persistence

[illegible]

Extended persistence



		1	2	2	3	3	3	inf	3	2	2	1	1	1
		1	2	12	3	13	23	c	c3	c2	c23	c1	c12	c13
1	1			1		1								
2	2			1										
2	12												1	1
3	3					1			1					
3	13													1
3	23										1			1
inf	c								1					
3	c3										1			
2	c2										1		1	
2	c23													
1	c1												1	
1	c12													
1	c13													

Persistence pairing

(2, 12) Ord 0 (1, ∅) Ord 0
(3, 13) Ord 0

(c, c3) Rel
(c2, c23) Rel
(c1, c12) Rel

(23, c13) Ext 1

Persistence diagram

(2, 2) 0 (1, ∞) Ord 0
(3, 3) 0

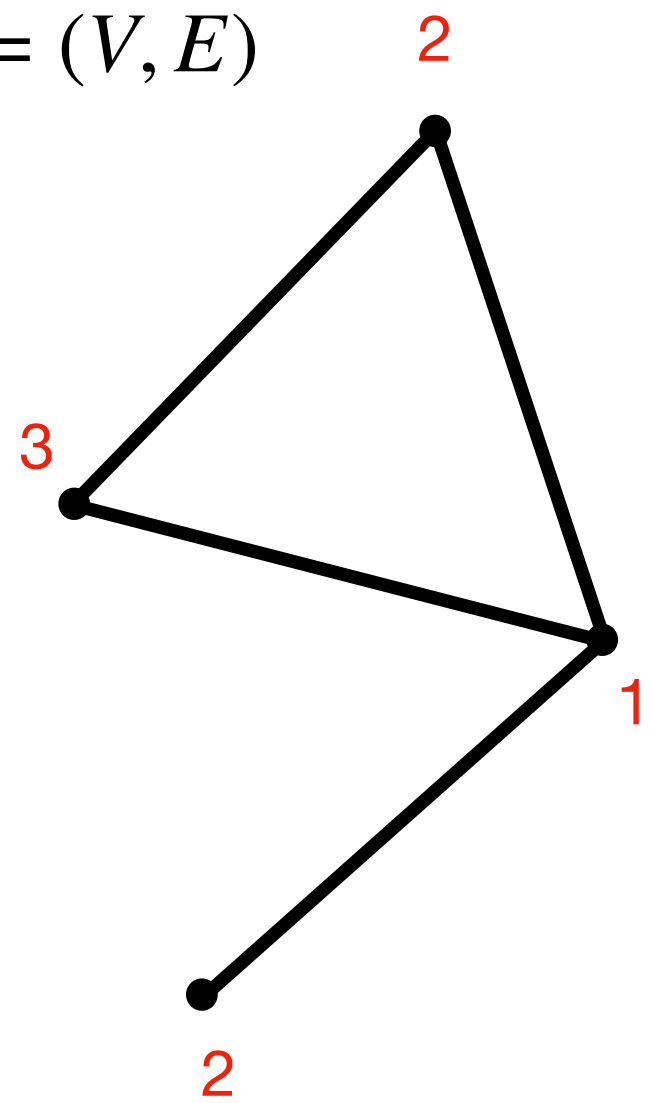
(inf, 3) Rel
(2, 2) Rel
(1, 1) Rel

(3, 1) Ext 1

Vertex function-induced filtration

Graph

$G = (V, E)$



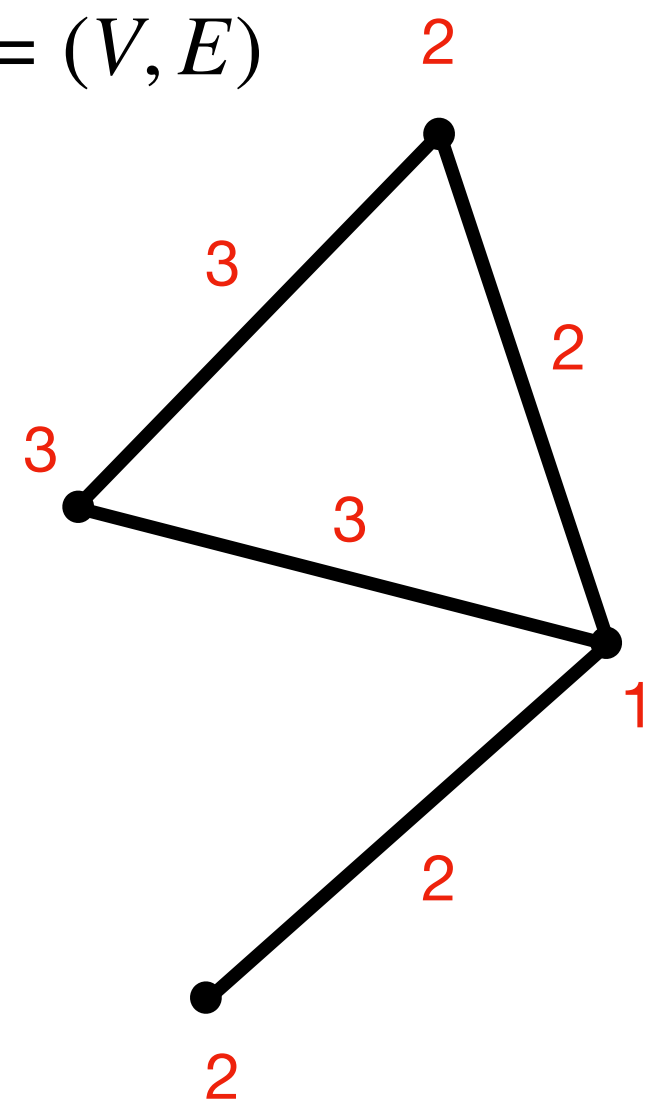
Vertex function

$g : V \rightarrow \mathbb{R}$

Vertex function-induced filtration

Graph

$G = (V, E)$



Vertex function

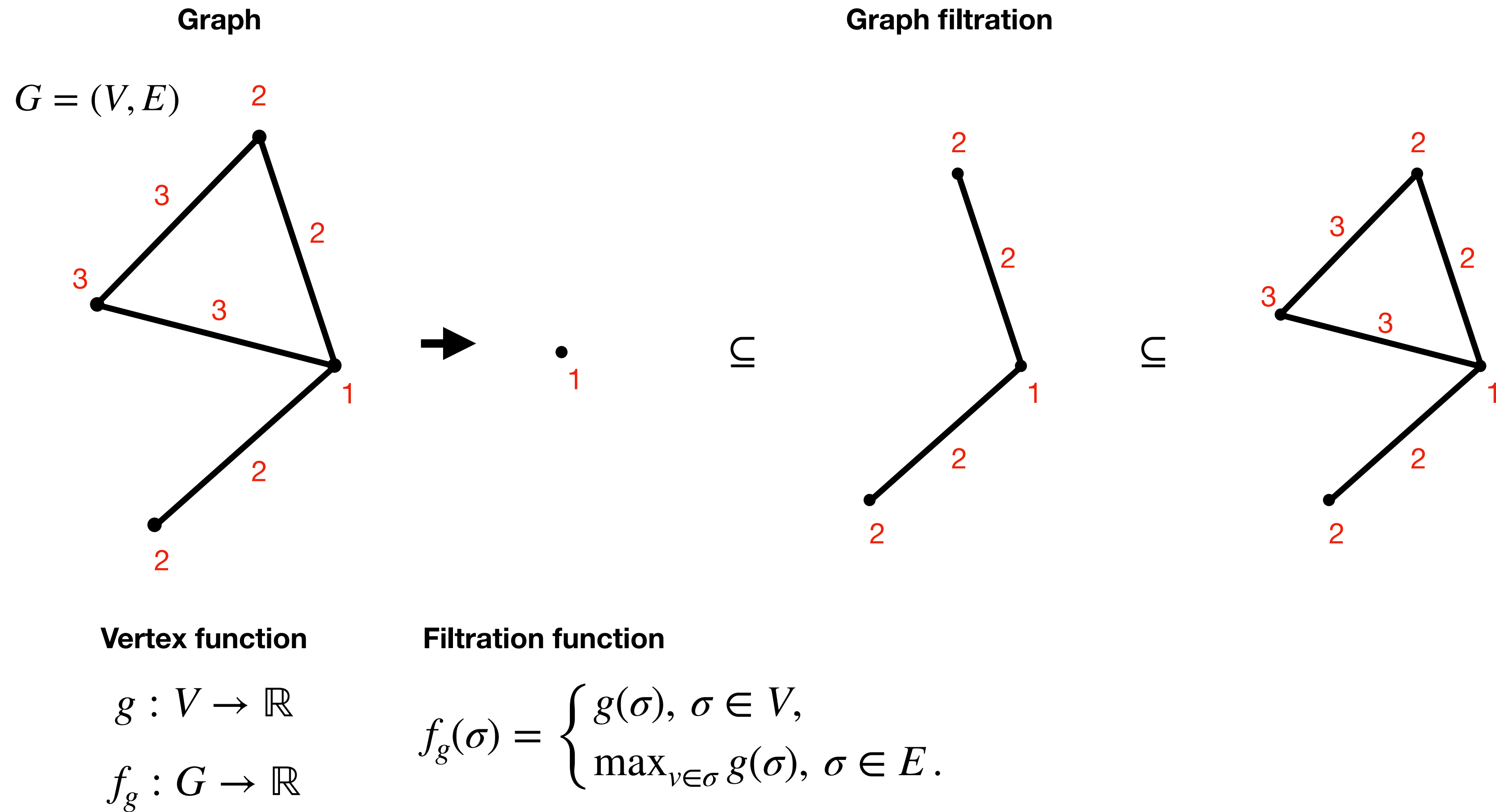
$$g : V \rightarrow \mathbb{R}$$

$$f_g : G \rightarrow \mathbb{R}$$

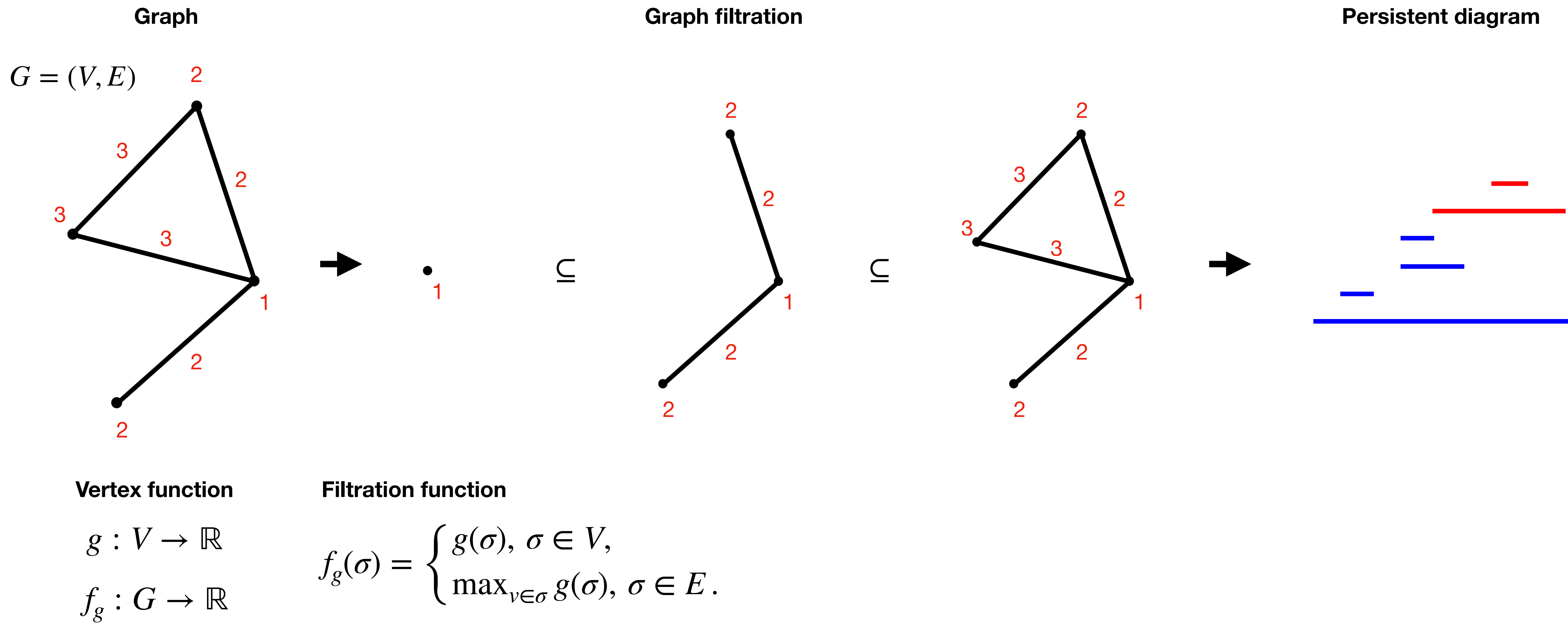
Filtration function

$$f_g(\sigma) = \begin{cases} g(\sigma), & \sigma \in V, \\ \max_{v \in \sigma} g(\sigma), & \sigma \in E. \end{cases}$$

Vertex function-induced filtration

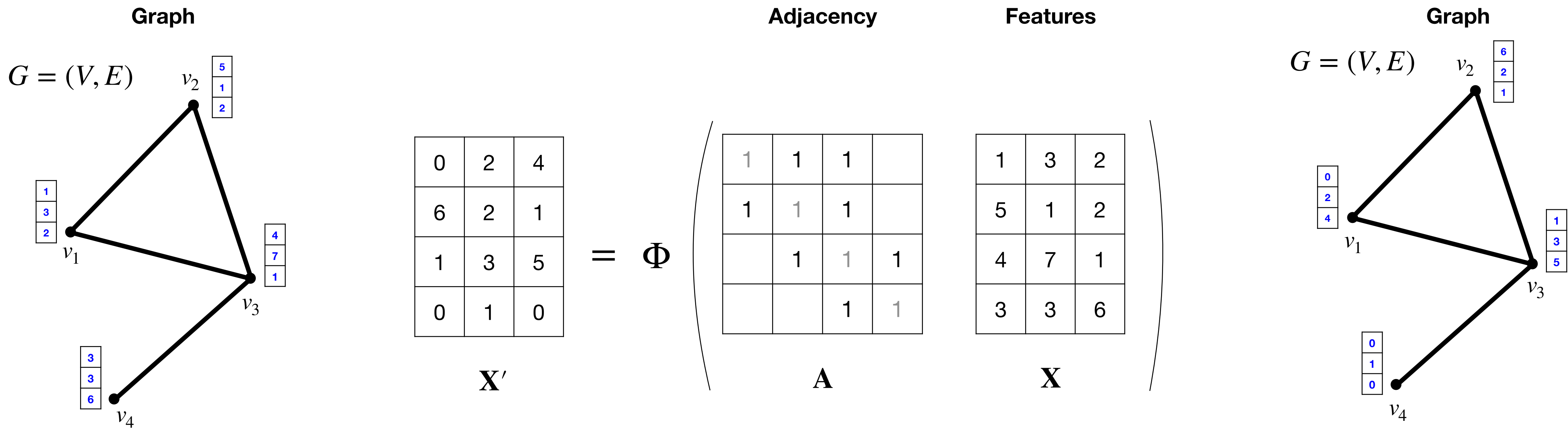


Vertex function-induced filtration



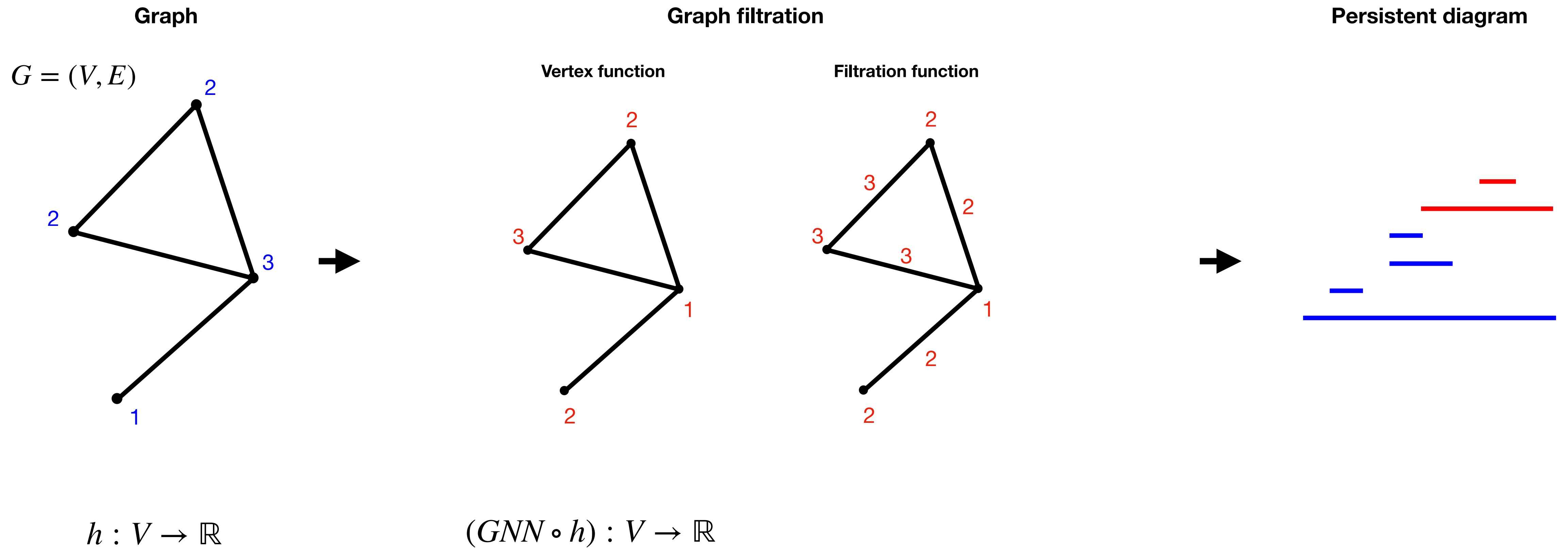
Message passing

Graph neural network

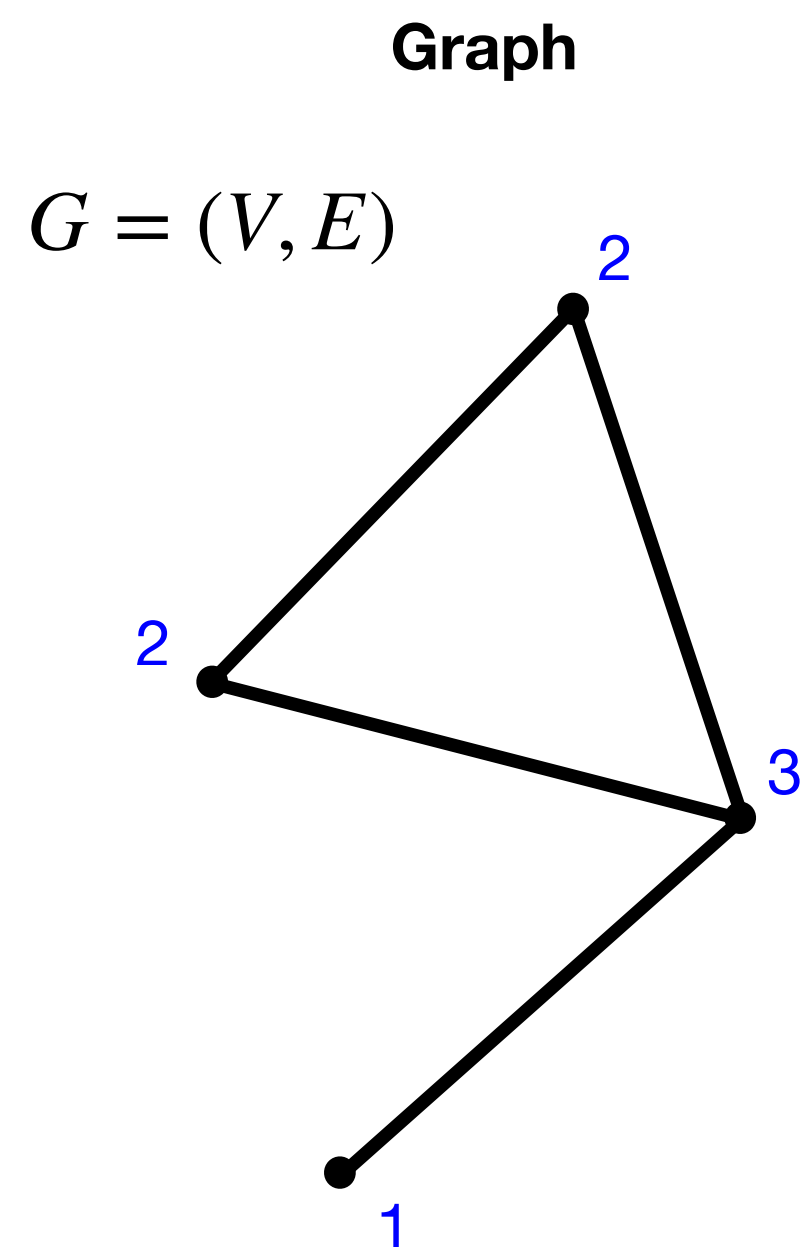


$$x'(v) = \psi \left(\sum_{u \in N(v)} x(u) \right)$$

Filtration learning



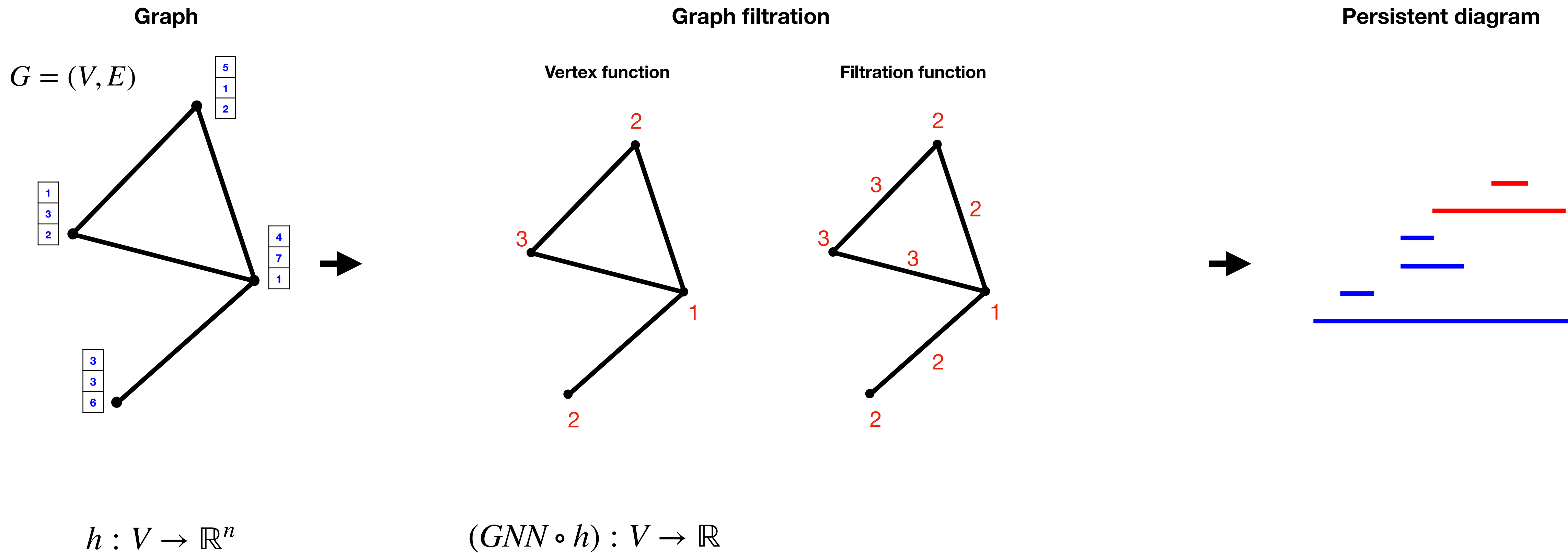
Filtration learning



Method	REDDIT-BINARY	REDDIT-MULTI-5K	IMDB-BINARY	IMDB-MULTI
	<i>Initial node features: uninformative</i>		<i>Initial node features: $l(v) = \deg(v)$</i>	
PH-only	90.3 $\pm$ 2.6	55.7 $\pm$ 2.1	68.9 $\pm$ 3.5	46.1 $\pm$ 4.2
1-GIN (GFL)	90.2 $\pm$ 2.8	55.7 $\pm$ 2.9	74.5 $\pm$ 4.6	49.7 $\pm$ 2.9
1-GIN (SUM) (Xu et al., 2019)	81.2 $\pm$ 5.4	51.0 $\pm$ 2.2	73.5 $\pm$ 3.8	50.3 $\pm$ 2.6
1-GIN (SP) (Zhang et al., 2018a)	76.8 $\pm$ 3.6	48.5 $\pm$ 1.8	73.0 $\pm$ 4.0	50.5 $\pm$ 2.1
Baseline (Zaheer et al., 2017)	77.5 $\pm$ 4.2	45.7 $\pm$ 1.4	72.7 $\pm$ 4.6	49.9 $\pm$ 4.0
<i>State-of-the-Art (NN)</i>				
DCNN (Wang et al., 2018)	n/a	n/a	49.1	33.5
PatchySAN (Niepert et al., 2016)	86.3	49.1	71.0	45.2
DGCNN (Zhang et al., 2018a)	n/a	n/a	70.0	47.8
1-2-3-GNN (Morris et al., 2019)	n/a	n/a	74.2	49.5
5-GIN (SUM) (Xu et al., 2019)	88.9	54.0	74.0	48.8

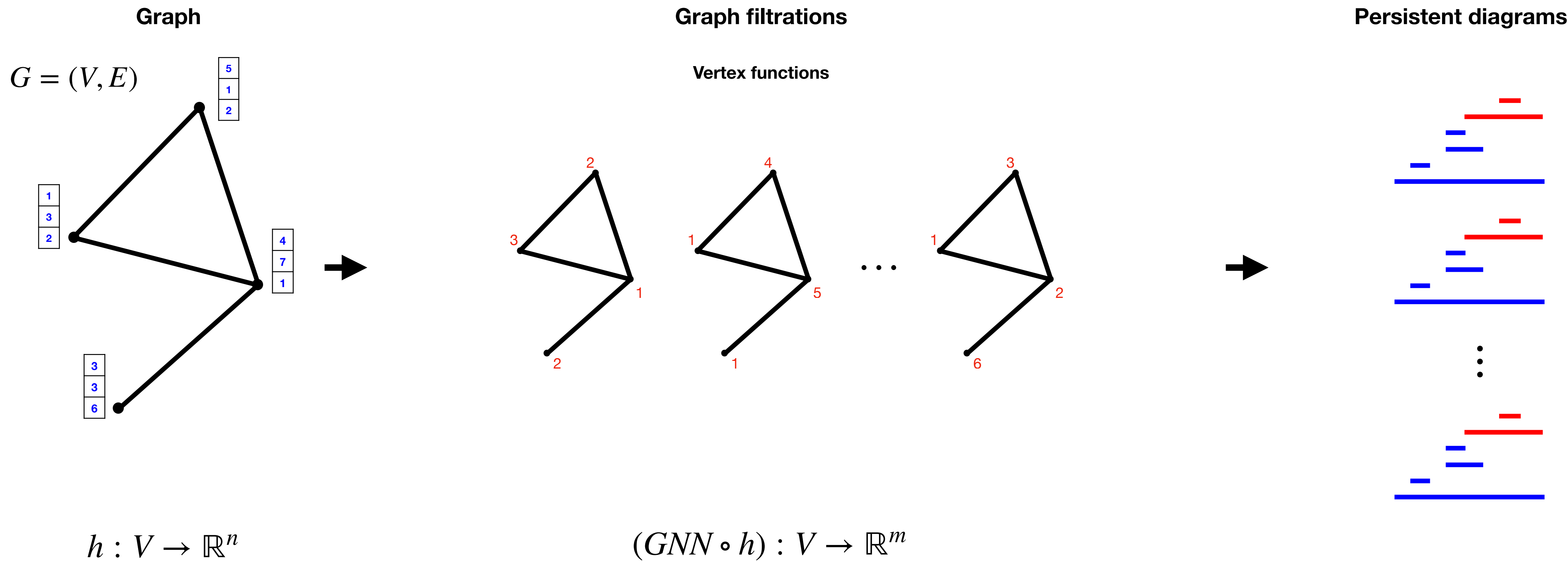
Filtration learning

Variants



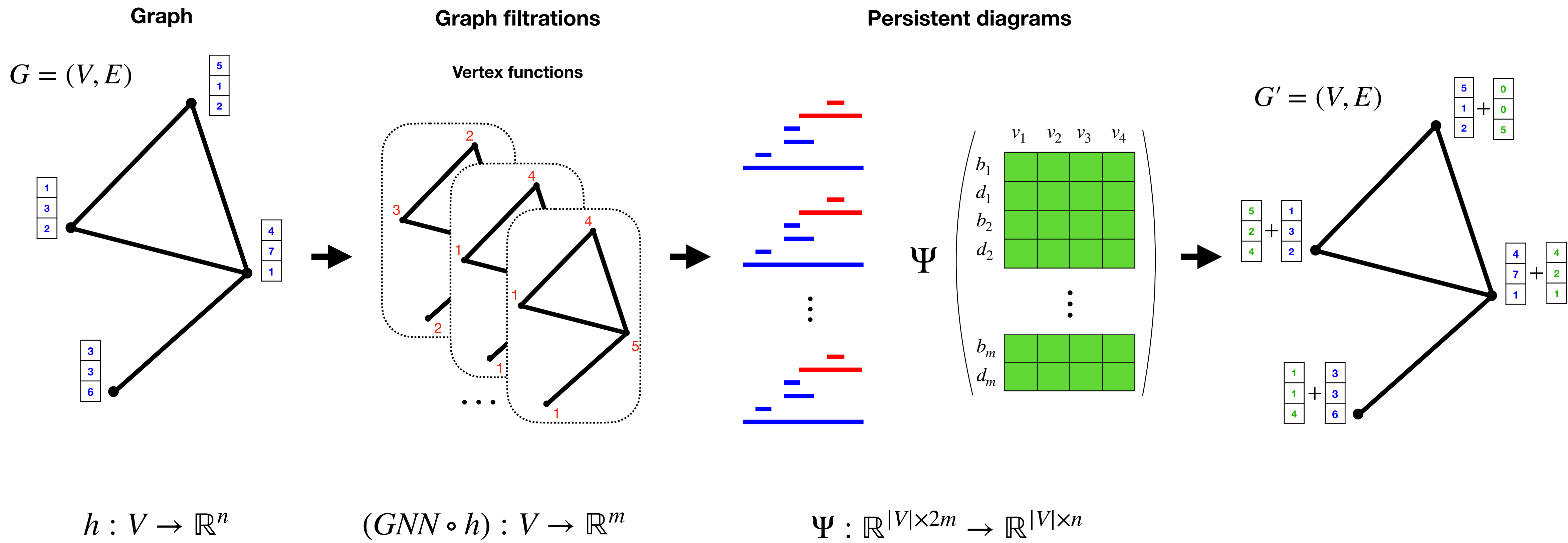
Filtration learning

Variants



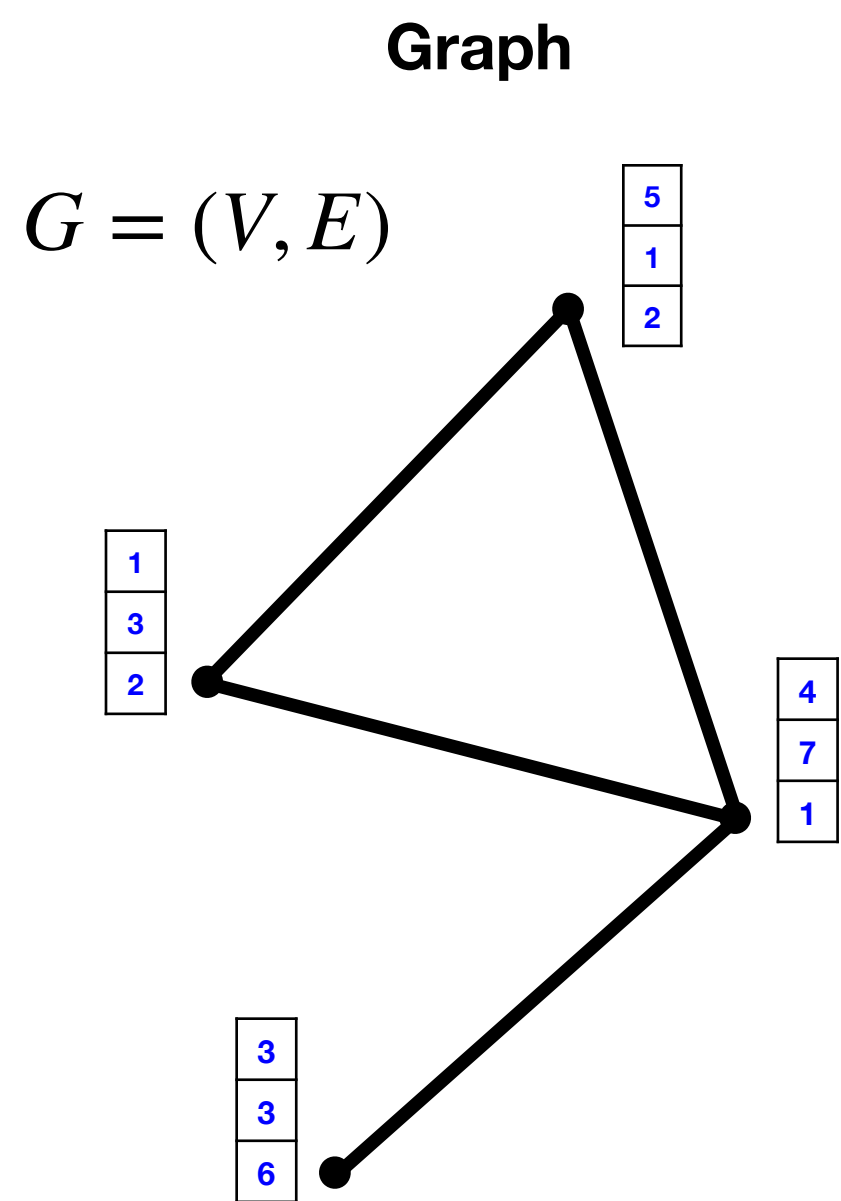
Filtration learning

Variants



Filtration learning

Variants

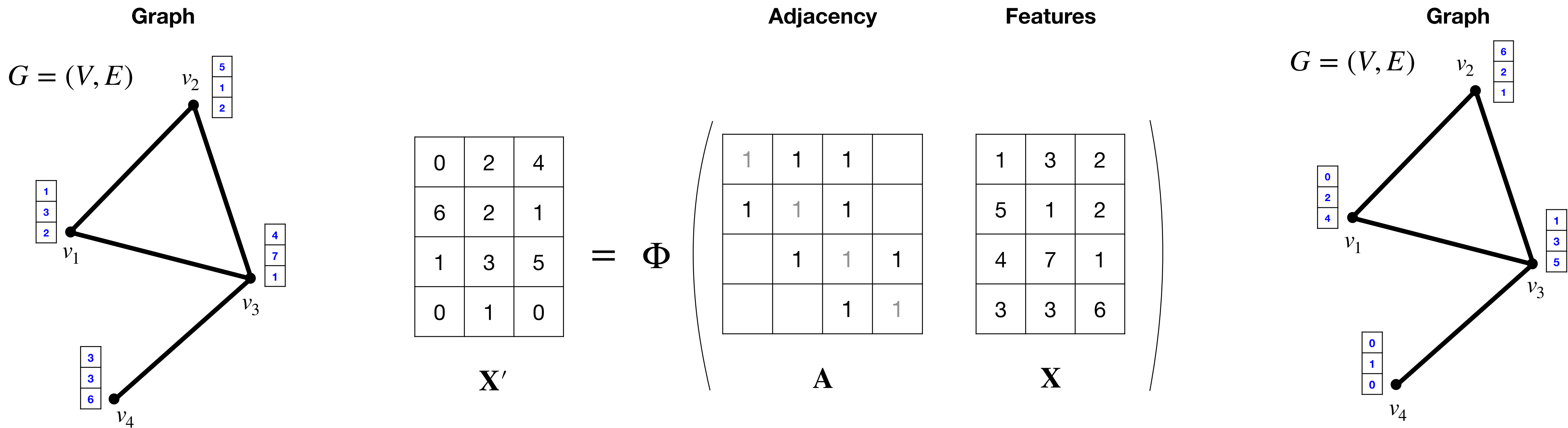


$$h : V \rightarrow \mathbb{R}^n$$

METHOD	Graph classification							Node classification	
	CIFAR-10	DD	ENZYMES	MNIST	PROTEINS-FULL	IMDB-B	REDDIT-B	CLUSTER	PATTERN
GAT-4	64.2 ± 0.4	75.9 ± 3.8	68.5 ± 5.2	95.5 ± 0.2	76.3 ± 2.4	—	—	57.7 ± 0.3	75.8 ± 1.8
GATED-GCN-4	67.3 ± 0.3	72.9 ± 2.1	65.7 ± 4.9	97.3 ± 0.1	76.4 ± 2.9	—	—	60.4 ± 0.4	84.5 ± 0.1
GIN-4	55.5 ± 1.5	71.9 ± 3.9	65.3 ± 6.8	96.5 ± 0.3	74.1 ± 3.4	72.9 ± 4.7	89.8 ± 2.2	58.4 ± 0.2	85.6 ± 0.0
WL	—	77.7 ± 2.0	54.3 ± 0.9	—	73.1 ± 0.5	71.2 ± 0.5	78.0 ± 0.6	—	—
WL-OA	—	77.8 ± 1.2	58.9 ± 0.9	—	73.5 ± 0.9	74.0 ± 0.7	87.6 ± 0.3	—	—
GCN-4 (baseline)	54.2 ± 1.5	72.8 ± 4.1	65.8 ± 4.6	90.0 ± 0.3	76.1 ± 2.4	68.6 ± 4.9	92.8 ± 1.7	57.0 ± 0.9	85.5 ± 0.4
GCN-3-TOGL-1	61.7 ± 1.0	73.2 ± 4.7	53.0 ± 9.2	95.5 ± 0.2	76.0 ± 3.9	72.0 ± 2.3	89.4 ± 2.2	60.4 ± 0.2	86.6 ± 0.1
GCN-3-TOGL-1 (static)	62.1 ± 0.5	71.0 ± 2.8	49.8 ± 7.0	95.4 ± 0.1	75.7 ± 3.6	72.8 ± 5.4	92.1 ± 1.6	60.5 ± 0.2	85.6 ± 0.1
GIN-3-TOGL-1	61.3 ± 0.4	75.2 ± 4.2	43.8 ± 7.9	96.1 ± 0.1	73.6 ± 4.8	74.2 ± 4.2	89.7 ± 2.5	60.4 ± 0.2	86.7 ± 0.1
GIN-3-TOGL-1 (static)	61.8 ± 0.6	72.2 ± 5.3	43.3 ± 8.3	96.4 ± 0.1	74.7 ± 3.1	73.8 ± 2.4	89.1 ± 4.4	60.6 ± 0.3	85.5 ± 0.1
GAT-3-TOGL-1	52.8 ± 3.4	73.7 ± 2.9	51.5 ± 7.3	0.0 ± 0.0	75.2 ± 3.9	70.8 ± 8.0	82.5 ± 8.7	58.4 ± 3.7	59.6 ± 3.3
GAT-3-TOGL-1 (static)	50.8 ± 3.6	72.8 ± 3.4	55.2 ± 9.1	96.2 ± 0.3	74.4 ± 2.5	68.7 ± 9.4	70.1 ± 9.9	58.7 ± 2.2	64.5 ± 14.2

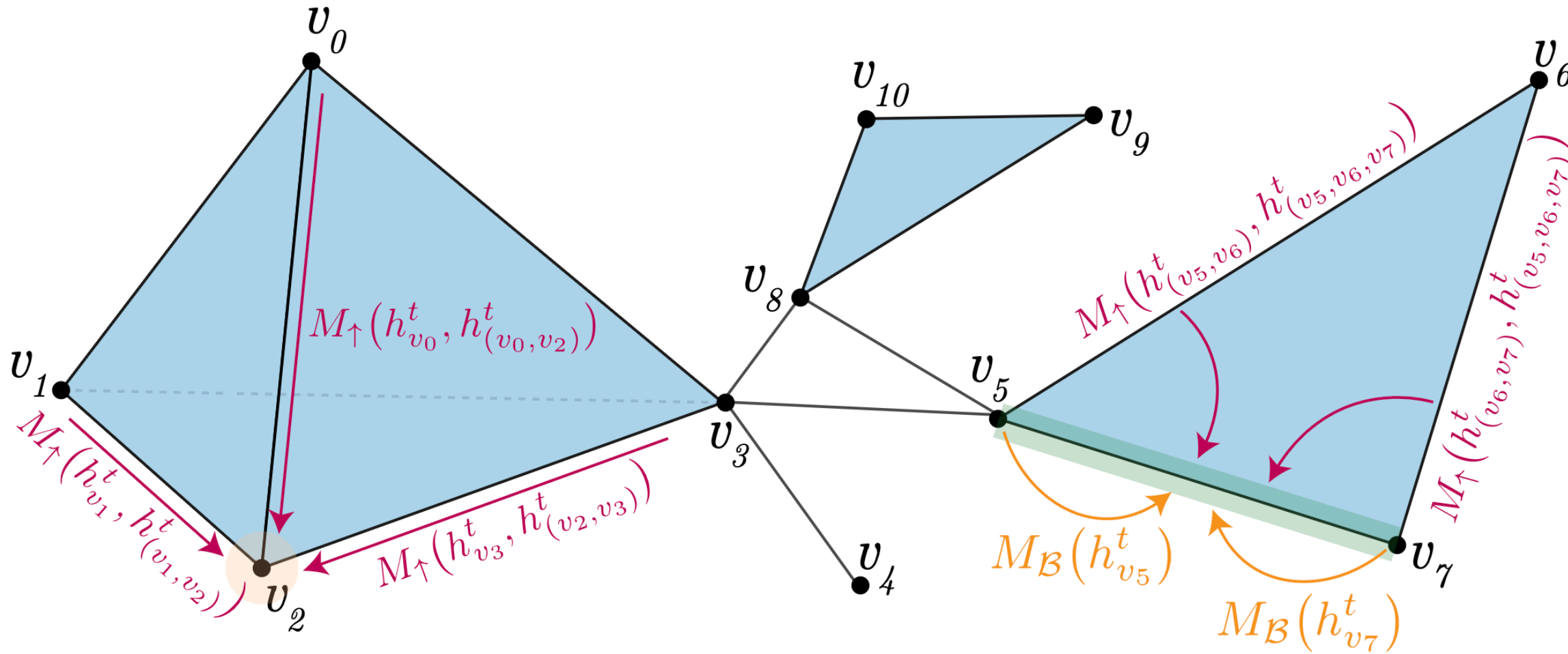
Message passing

Graph neural network



$$x'(v) = \psi \left(\sum_{u \in N(v)} x(u) \right)$$

Simplicial message passing



Definition 4. Consider a simplex $\sigma \in \mathcal{K}$. Four types of adjacent simplices can be defined:

1. Boundary adjacencies $\mathcal{B}(\sigma) = \{\tau \mid \tau \prec \sigma\}$.
2. Co-boundary adjacencies $\mathcal{C}(\sigma) = \{\tau \mid \sigma \prec \tau\}$.
3. Lower-adjacencies $\mathcal{N}_{\downarrow}(\sigma) = \{\tau \mid \exists \delta, \delta \prec \tau \wedge \delta \prec \sigma\}$
4. Upper-adjacencies $\mathcal{N}_{\uparrow}(\sigma) = \{\tau \mid \exists \delta, \tau \prec \delta \wedge \sigma \prec \delta\}$

MPSN We propose a message passing model using the following message passing operations based on the four types of messages discussed in the previous section. For a simplex σ in a complex $\mathcal{K}$ we have:

$$m_{\mathcal{B}}^{t+1}(\sigma) = \text{AGG}_{\tau \in \mathcal{B}(\sigma)} \left(M_{\mathcal{B}}(h_{\sigma}^t, h_{\tau}^t) \right) \quad (2)$$

$$m_{\mathcal{C}}^{t+1}(\sigma) = \text{AGG}_{\tau \in \mathcal{C}(\sigma)} \left(M_{\mathcal{C}}(h_{\sigma}^t, h_{\tau}^t) \right) \quad (3)$$

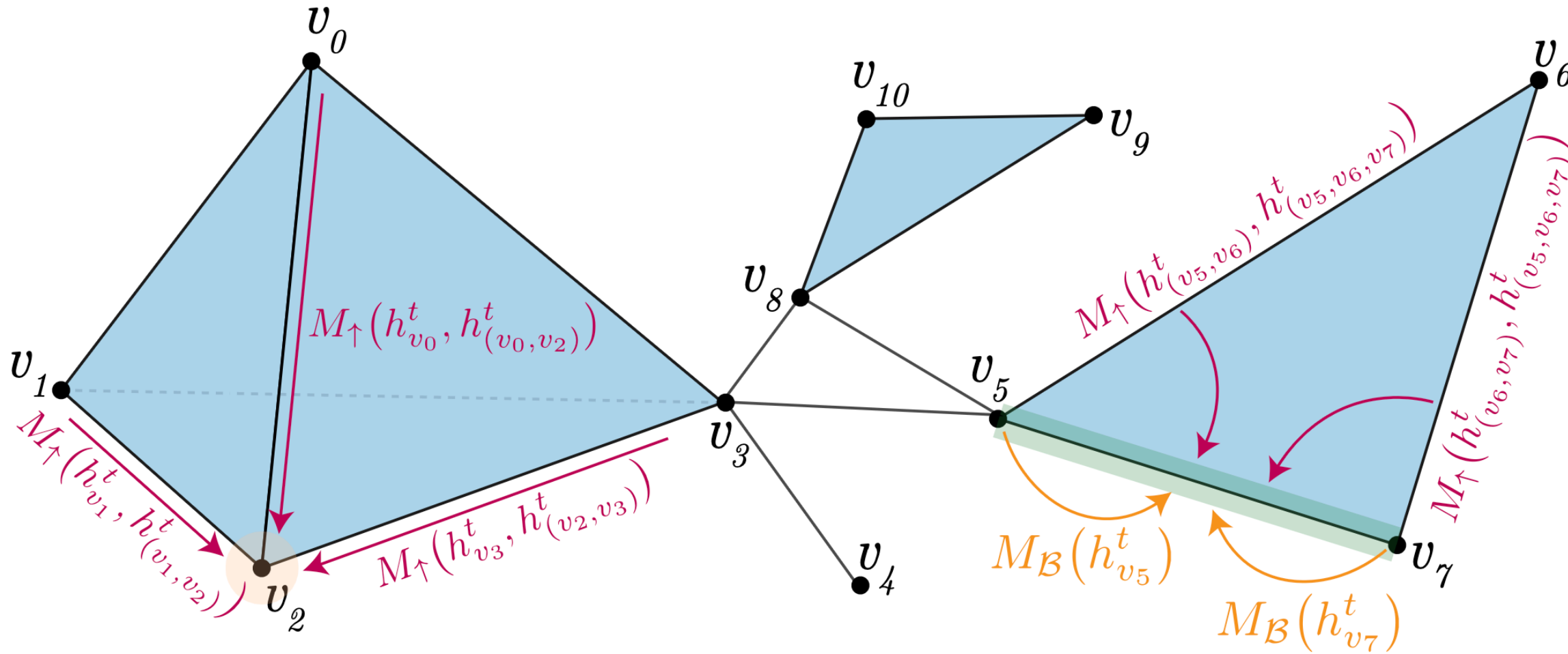
$$m_{\downarrow}^{t+1}(\sigma) = \text{AGG}_{\tau \in \mathcal{N}_{\downarrow}(\sigma)} \left(M_{\downarrow}(h_{\sigma}^t, h_{\tau}^t, h_{\sigma \cap \tau}^t) \right) \quad (4)$$

$$m_{\uparrow}^{t+1}(\sigma) = \text{AGG}_{\tau \in \mathcal{N}_{\uparrow}(\sigma)} \left(M_{\uparrow}(h_{\sigma}^t, h_{\tau}^t, h_{\sigma \cup \tau}^t) \right). \quad (5)$$

Then, the update operation takes into account these four types of incoming messages and the previous colour of the simplex:

$$h_{\sigma}^{t+1} = U \left(h_{\sigma}^t, m_{\mathcal{B}}^t(\sigma), m_{\mathcal{C}}^t(\sigma), m_{\downarrow}^{t+1}(\sigma), m_{\uparrow}^{t+1}(\sigma) \right). \quad (6)$$

Simplicial message passing



Definition 4. Consider a simplex $\sigma \in \mathcal{K}$. Four types of adjacent simplices can be defined:

1. Boundary adjacencies $\mathcal{B}(\sigma) = \{\tau \mid \tau \prec \sigma\}$.
2. Co-boundary adjacencies $\mathcal{C}(\sigma) = \{\tau \mid \sigma \prec \tau\}$.
3. Lower-adjacencies $\mathcal{N}_{\downarrow}(\sigma) = \{\tau \mid \exists \delta, \delta \prec \tau \wedge \delta \prec \sigma\}$
4. Upper-adjacencies $\mathcal{N}_{\uparrow}(\sigma) = \{\tau \mid \exists \delta, \tau \prec \delta \wedge \sigma \prec \delta\}$

Vertex adjacency
in graphs

MPSN We propose a message passing model using the following message passing operations based on the four types of messages discussed in the previous section. For a simplex σ in a complex $\mathcal{K}$ we have:

$$m_{\mathcal{B}}^{t+1}(\sigma) = \text{AGG}_{\tau \in \mathcal{B}(\sigma)} \left(M_{\mathcal{B}}(h_{\sigma}^t, h_{\tau}^t) \right) \quad (2)$$

$$m_{\mathcal{C}}^{t+1}(\sigma) = \text{AGG}_{\tau \in \mathcal{C}(\sigma)} \left(M_{\mathcal{C}}(h_{\sigma}^t, h_{\tau}^t) \right) \quad (3)$$

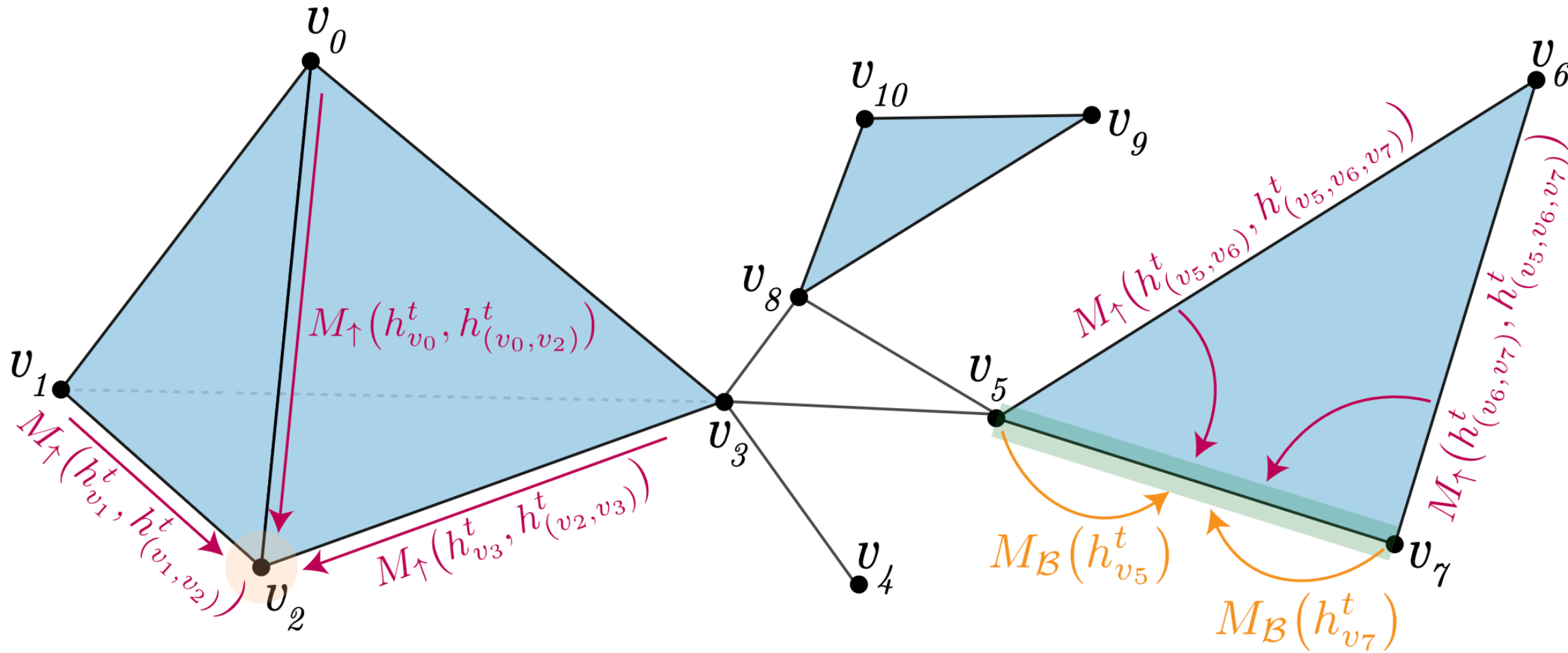
$$m_{\downarrow}^{t+1}(\sigma) = \text{AGG}_{\tau \in \mathcal{N}_{\downarrow}(\sigma)} \left(M_{\downarrow}(h_{\sigma}^t, h_{\tau}^t, h_{\sigma \cap \tau}^t) \right) \quad (4)$$

$$m_{\uparrow}^{t+1}(\sigma) = \text{AGG}_{\tau \in \mathcal{N}_{\uparrow}(\sigma)} \left(M_{\uparrow}(h_{\sigma}^t, h_{\tau}^t, h_{\sigma \cup \tau}^t) \right). \quad (5)$$

Then, the update operation takes into account these four types of incoming messages and the previous colour of the simplex:

$$h_{\sigma}^{t+1} = U \left(h_{\sigma}^t, m_{\mathcal{B}}^t(\sigma), m_{\mathcal{C}}^t(\sigma), m_{\downarrow}^{t+1}(\sigma), m_{\uparrow}^{t+1}(\sigma) \right). \quad (6)$$

Simplicial message passing



Definition 4. Consider a simplex $\sigma \in \mathcal{K}$. Four types of adjacent simplices can be defined:

1. Boundary adjacencies $\mathcal{B}(\sigma) = \{\tau \mid \tau \prec \sigma\}$.
2. Co-boundary adjacencies $\mathcal{C}(\sigma) = \{\tau \mid \sigma \prec \tau\}$.
3. Lower-adjacencies $\mathcal{N}_{\downarrow}(\sigma) = \{\tau \mid \exists \delta, \delta \prec \tau \wedge \delta \prec \sigma\}$
4. Upper-adjacencies $\mathcal{N}_{\uparrow}(\sigma) = \{\tau \mid \exists \delta, \tau \prec \delta \wedge \sigma \prec \delta\}$

Dataset	Proteins	NCI1	IMDB-B	IMDB-M	RDT-B	RDT-M5K
Avg Δ	27.4	0.05	392.0	305.9	24.8	21.8
Med Δ	21.0	0.0	119.5	56.0	11.0	11.0
RWK	59.6 $\pm$ 0.1	>3 days	N/A	N/A	N/A	N/A
GK (k=3)	71.4 $\pm$ 0.31	62.5 $\pm$ 0.3	N/A	N/A	N/A	N/A
PK	73.7 $\pm$ 0.7	82.5 $\pm$ 0.5	N/A	N/A	N/A	N/A
WL kernel	75.0 $\pm$ 3.1	86.0 $\pm$ 1.8	73.8 $\pm$ 3.9	50.9 $\pm$ 3.8	81.0 $\pm$ 3.1	52.5 $\pm$ 2.1
DCNN	61.3 $\pm$ 1.6	56.6 $\pm$ 1.0	49.1 $\pm$ 1.4	33.5 $\pm$ 1.4	N/A	N/A
DGCNN	75.5 $\pm$ 0.9	74.4 $\pm$ 0.5	70.0 $\pm$ 0.9	47.8 $\pm$ 0.9	N/A	N/A
IGN	76.6 $\pm$ 5.5	74.3 $\pm$ 2.7	72.0 $\pm$ 5.5	48.7 $\pm$ 3.4	N/A	N/A
GIN	76.2 $\pm$ 2.8	82.7 $\pm$ 1.7	75.1 $\pm$ 5.1	52.3 $\pm$ 2.8	92.4 $\pm$ 2.5	57.5 $\pm$ 1.5
PPGNs	77.2 $\pm$ 4.7	83.2 $\pm$ 1.1	73.0 $\pm$ 5.8	50.5 $\pm$ 3.6	N/A	N/A
Natural GN	71.7 $\pm$ 1.0	82.4 $\pm$ 1.3	73.5 $\pm$ 2.0	51.3 $\pm$ 1.5	N/A	N/A
GSN	76.6 $\pm$ 5.0	83.5 $\pm$ 2.0	77.8 $\pm$ 3.3	54.3 $\pm$ 3.3	N/A	N/A
SIN (Ours)	76.5 $\pm$ 3.4	82.8 $\pm$ 2.2	75.6 $\pm$ 3.2	52.5 $\pm$ 3.0	92.2 $\pm$ 1.0	57.3 $\pm$ 1.6

Simplicial message passing

Neighborhood Structures

Boundary Adjacent Neighborhood

$\mathcal{B}(y) = \{x \mid x \prec y\}$:
The set of y -connected x cells of next lower rank.
The neighborhood is specified with the *boundary matrix* B_r . Example: The set of nodes x connected to edge y .

Co-Boundary Adjacent Neighborhood

$\mathcal{C}(y) = \{x \mid y \prec x\}$:
The set of y -connected x cells of next higher rank.
The neighborhood is specified with the *co-boundary matrix* B_r^T . Example: The set of edges x connected to node y .

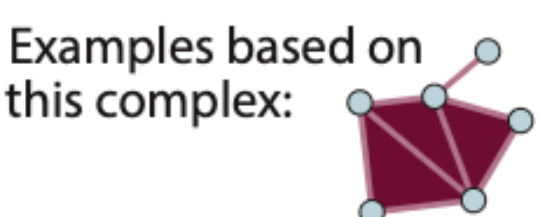
Lower Adjacent Neighborhood

$\mathcal{L}_{\downarrow}(y) = \{x \mid \exists z \text{ s.t. } z \prec y \text{ and } z \prec x\}$:
The set of x cells that share a boundary z with y . The neighborhood is specified with either the *lower Laplacian matrix* $L_{\downarrow,r} = B_r B_r^T$ or the *lower adjacency matrix* $A_{\downarrow,r} = D_{\downarrow} - L_{\downarrow,r}$. Example: the set of edges x that connect to any of the nodes z that touch edge y .

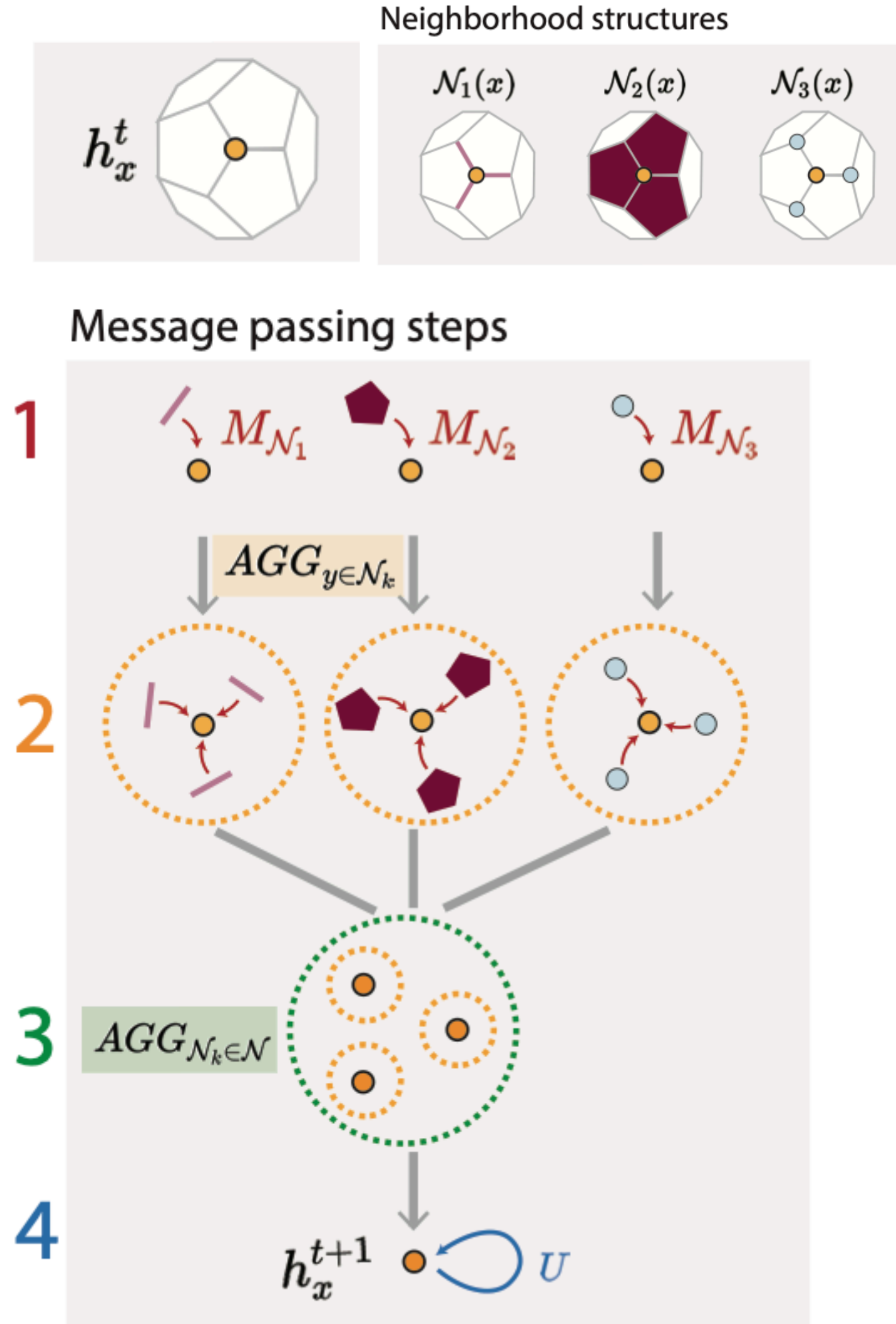
Upper Adjacent Neighborhood

$\mathcal{L}_{\uparrow}(y) = \{x \mid \exists z \text{ s.t. } y \prec z \text{ and } x \prec z\}$:
The set of x cells that share a co-boundary z with y . The neighborhood is specified with either the *upper Laplacian matrix* $L_{\uparrow,r} = B_{r+1}^T B_{r+1}$ or the *upper adjacency matrix* $A_{\uparrow,r} = D_{\uparrow} - L_{\uparrow,r}$. Example: The set of nodes x that touch any of the edges z that touch node y .

Neighborhood Structure	Neighborhood Matrix	Examples on a complex	
		Ex. 1	Ex. 2
Boundary All y -connected x cells of next lower rank	$\textcircled{r} \xrightarrow{B_r} \textcircled{r-1}$		
Co-boundary All y -connected x cells of next higher rank	$\textcircled{r-1} \xrightarrow{B_r^T} \textcircled{r}$		
Lower adjacent All x cells sharing a boundary z with y	$\textcircled{r} \xrightarrow{L_{\downarrow,r}} \textcircled{r-1} \xrightarrow{B_r^T} \textcircled{r}$		
	$A_{\downarrow,r} = D_{\downarrow} - L_{\downarrow,r}$ $\textcircled{r} \xrightarrow{A_{\downarrow,r}} \textcircled{r-1} \xrightarrow{B_r^T} \textcircled{r}$		
Upper adjacent All x cells sharing a co-boundary z with y	$\textcircled{r} \xrightarrow{L_{\uparrow,r}} \textcircled{r+1} \xrightarrow{B_{r+1}} \textcircled{r}$		
	$A_{\uparrow,r} = D_{\uparrow} - L_{\uparrow,r}$ $\textcircled{r} \xrightarrow{A_{\uparrow,r}} \textcircled{r+1} \xrightarrow{B_{r+1}} \textcircled{r}$		



Simplicial message passing



1. Message (red): First, a message $m_{y \rightarrow x}^{(r' \rightarrow r)}$ travels from a r' -cell y to a r -cell x through a neighborhood k of x denoted $\mathcal{N}_k(x)$:

$$m_{y \rightarrow x}^{(r' \rightarrow r)} = M_{\mathcal{N}_k} \left(\mathbf{h}_x^{t,(r)}, \mathbf{h}_y^{t,(r')}, \Theta^t \right). \quad (2)$$

via the function $M_{\mathcal{N}_k}$ depicted in red in Figure 8. Here, $\mathbf{h}_x^{t,(r)}$ and $\mathbf{h}_y^{t,(r')}$ are features of dimension d_r and $d_{r'}$ on cells y and x respectively, and Θ^t are learnable parameters. In the simplest case, this step looks like a neighborhood matrix M propagating a feature $\mathbf{h}_y^{t,(r')}$ on r' -cell y to r -cell x as:

$$m_{y \rightarrow x}^{(r' \rightarrow r)} = M_{xy} \cdot \mathbf{h}_y^{t,(r')} \cdot \Theta^t, \quad (3)$$

where M_{xy} is the scalar entry of matrix M at the row corresponding to cell x and column corresponding to cell y and $m_{y \rightarrow x}^{(r' \rightarrow r)}$ and Θ is a $d_{r'} \times d_r$ matrix. If y is not in the neighborhood structure of x , then M_{xy} will be 0, and x cannot receive any message from y .

2. Within-Neighborhood Aggregation (orange): Next, messages are aggregated across all cells y belonging to the neighborhood $\mathcal{N}_k(x)$:

$$m_x^{(r' \rightarrow r)} = AGG_{y \in \mathcal{N}_k(x)} m_{y \rightarrow x}^{(r' \rightarrow r)}, \quad (4)$$

resulting in the *within-neighborhood aggregated message* $m_x^{(r' \rightarrow r)}$. Here, AGG is an aggregation function, depicted in orange in Figure 8, analogous to pooling in standard convolutional networks.

3. Between-Neighborhood Aggregation (green): Then, messages are aggregated across neighborhoods in a neighborhood set $\mathcal{N}$:

$$m_x^{(r)} = AGG_{\mathcal{N}_k \in \mathcal{N}} m_x^{(r' \rightarrow r)}, \quad (5)$$

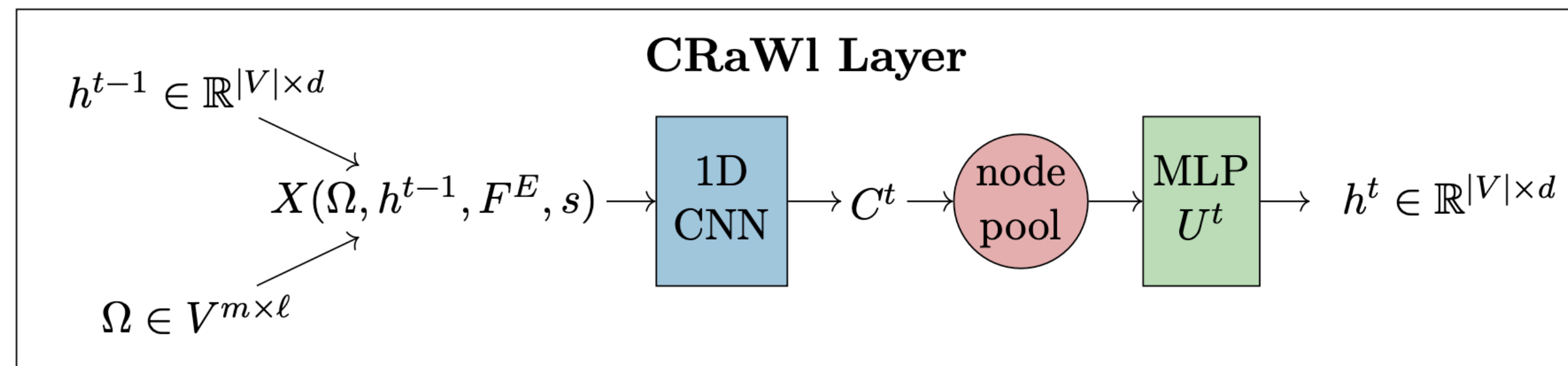
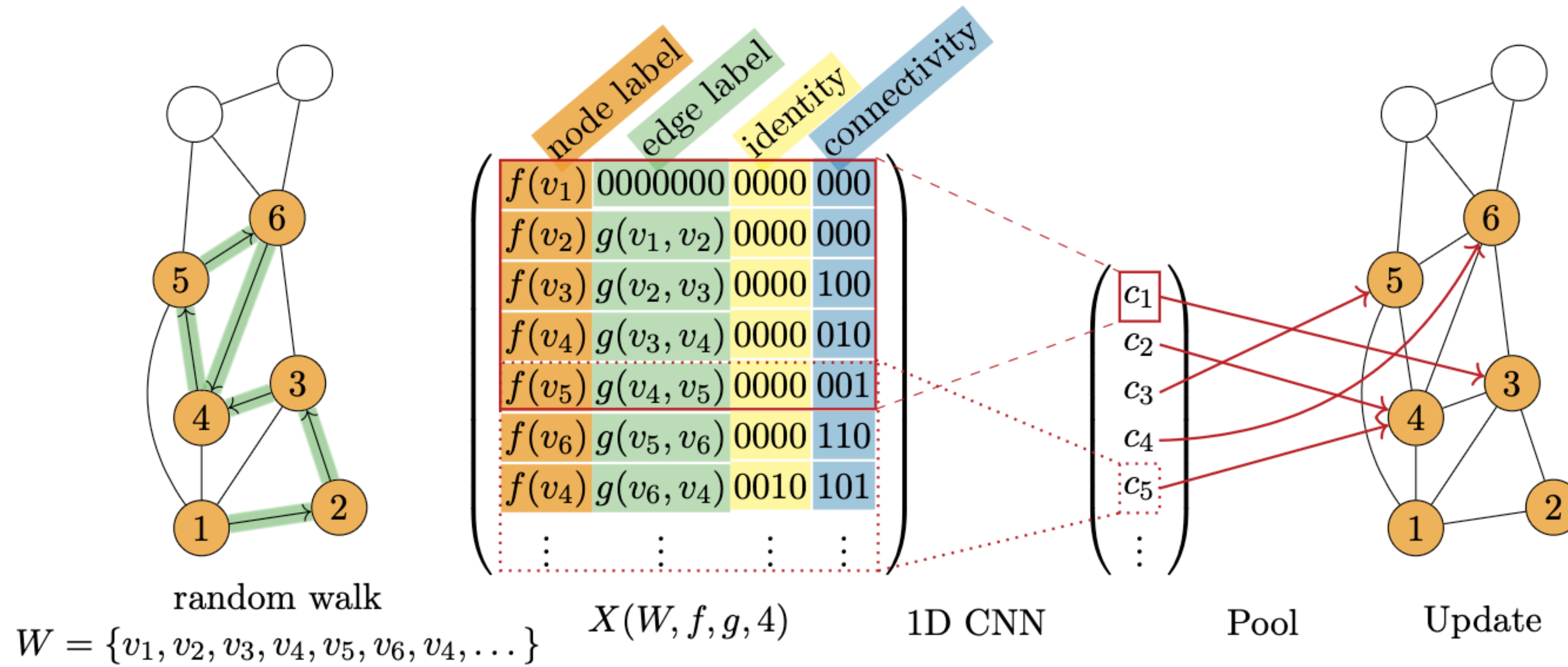
where AGG is a (potentially different) aggregation function depicted in green in Figure 8, and $m_x^{(r)}$ is the message received by cell x that triggers the update of its feature.

4. Update (blue): Finally, the feature on cell x is updated via a function U depicted in blue in Figure 8, which may depend on the previous feature $\mathbf{h}_x^{t,(r)}$ on cell x :

$$\mathbf{h}_x^{t+1,(r)} = U \left(\mathbf{h}_x^{t,(r)}, m_x^{(r)} \right), \quad (6)$$

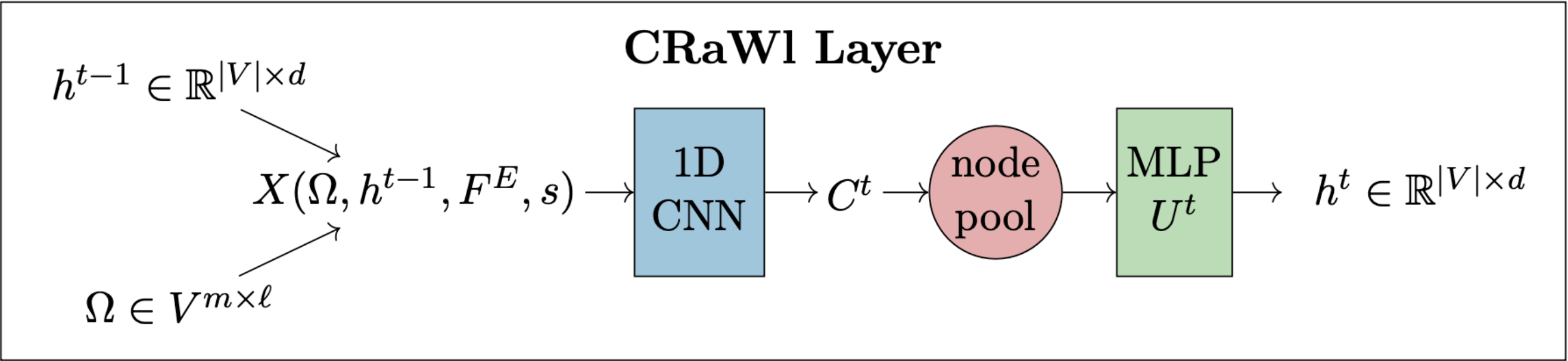
The result $\mathbf{h}_x^{t+1,(r)}$ is the updated feature on cell x that is input to layer $t + 1$.

Random walks



Random walks

METHOD		ZINC TEST MAE	MNIST TEST ACC (%)	CIFAR10 TEST ACC (%)	MOLPCBA TEST AP
LEADERBOARD	GIN	0.526 ± 0.051	96.485 ± 0.252	55.255 ± 1.527	0.2703 ± 0.0023
	GRAPHSAGE	0.398 ± 0.002	97.312 ± 0.097	65.767 ± 0.308	-
	GAT	0.384 ± 0.007	95.535 ± 0.205	64.223 ± 0.455	-
	GCN	0.367 ± 0.011	90.705 ± 0.218	55.710 ± 0.381	0.2483 ± 0.0037
	3WLGNN	0.303 ± 0.068	95.075 ± 0.961	59.175 ± 1.593	-
	GATEDGCN	0.214 ± 0.006	97.340 ± 0.143	67.312 ± 0.311	-
	PNA	0.142 ± 0.010	97.940 ± 0.120	70.350 ± 0.630	0.2838 ± 0.0035
	GINE+	-	-	-	0.2917 ± 0.0015
OUR	CRAWL	0.085 ± 0.004	97.944 ± 0.050	69.013 ± 0.259	0.2986 ± 0.0025



Random walks

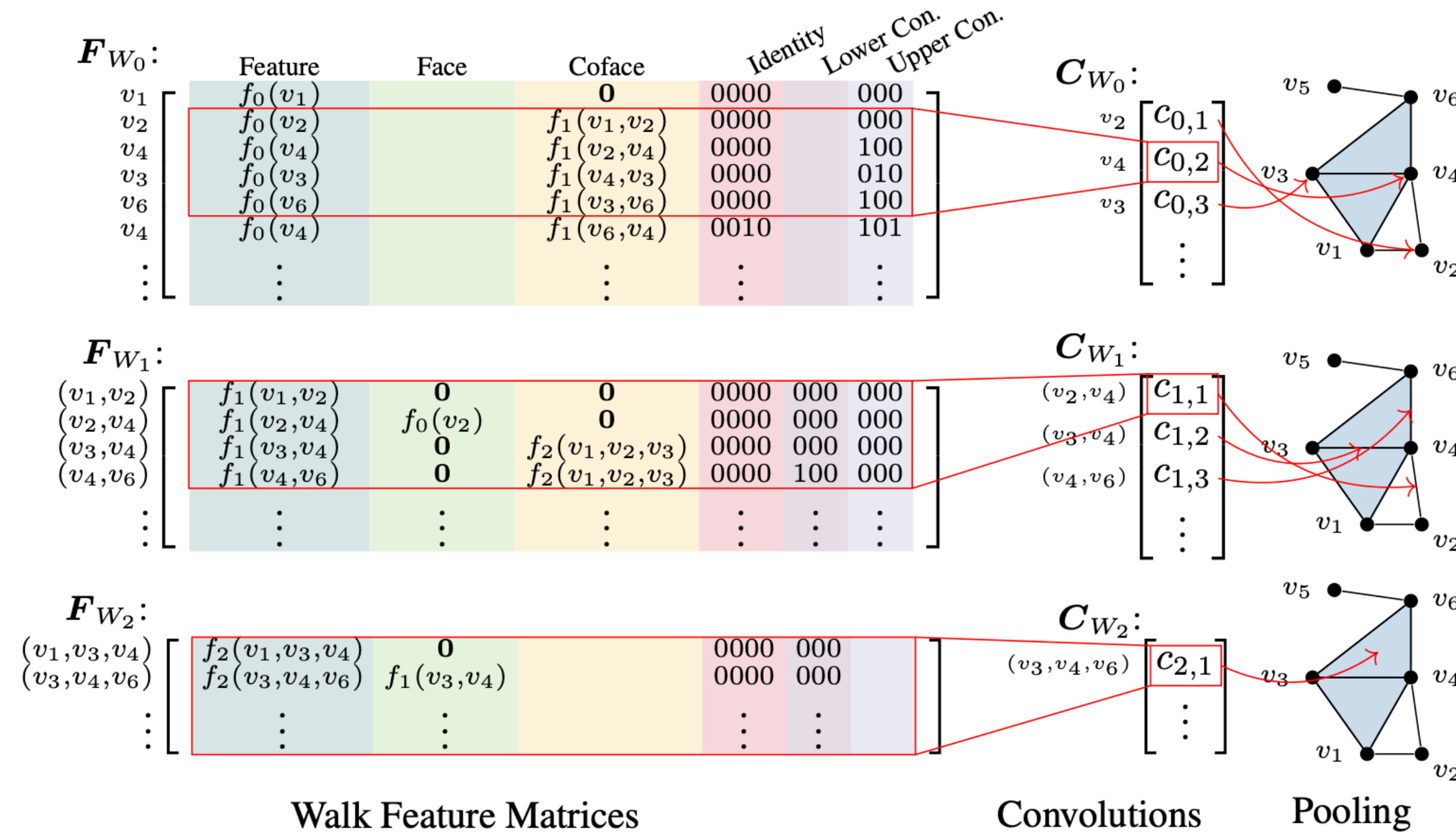


Figure 2: **Information flow in SCRaWI modules for simplex orders 0, 1, and 2.** Using the walks shown in Figure 1, we compute the feature matrices F_0 , F_1 , and F_2 based on simplex features f and a window size of $s = 4$. The feature matrices are convolved with a 1D CNN while keeping track of the central simplex within the window (values to the left of the convolution matrices). Convolved matrices are pooled into simplices grouped by the central simplices.

More formally, given a walk W_j of length ℓ on k -simplices, embedding functions $f_{k'} : X_{k'} \rightarrow \mathbb{R}^{d_{k'}}$ for $i \in \{k-1, k, k+1\}$, and a local window size $s > 0$, we define the walk feature matrix F_{W_j} as

$$F_{W_j} = \begin{bmatrix} F_{W_j}^{\rightarrow} & F_{W_j}^{\downarrow} & F_{W_j}^{\uparrow} & I_{W_j,s} & A_{W_j,s}^{\downarrow} & A_{W_j,s}^{\uparrow} \end{bmatrix} \in \mathbb{R}^{\ell \times (d_k + d_{k-1} + d_{k+1} + 3s - 2)}. \quad (5)$$

The first sub-matrix $F_{W_j} \in \mathbb{R}^{\ell \times d_k}$ records the features of the k -simplices appearing on the walk W :

$$(F_{W_j}^{\rightarrow})_{i,-} = f(v_i) \text{ for } i \in [\ell]. \quad (6)$$

The second and third sub-matrices $F_{W_j}^{\downarrow}$ and $F_{W_j}^{\uparrow}$ record the features of the faces and cofaces, respectively, that have been used in the random walk to get from the last simplex to the current one:

$$(F_{W_j}^{\downarrow})_{i,-} = \begin{cases} f(e_{i-1}), & \text{if } i > 0 \text{ and } |e_{i-1}| < k \\ \mathbf{0}_{d_{k-1}}, & \text{otherwise.} \end{cases} \quad (7)$$

$$(F_{W_j}^{\uparrow})_{i,-} = \begin{cases} f(e_{i-1}), & \text{if } i > 0 \text{ and } |e_{i-1}| > k \\ \mathbf{0}_{d_{k+1}}, & \text{otherwise.} \end{cases} \quad (8)$$

The remaining three sub-matrices $I_{W_j} \in \{0, 1\}^{\ell \times s}$, $A_{W_j}^{\downarrow} \in \{0, 1\}^{\ell \times (s-1)}$ and $A_{W_j}^{\uparrow} \in \{0, 1\}^{\ell \times (s-1)}$ record structural information in the form of local identity and local adjacency features:

$$(I_{W_j,s})_{i,i'} = \begin{cases} 1, & \text{if } i \geq i' \text{ and } v_i = v_{i'} \\ 0, & \text{otherwise.} \end{cases} \quad (9)$$

$$(A_{W_j,s}^{\downarrow})_{i,i'} = \begin{cases} 1, & \text{if } i > i' \text{ and } v_{i'-i} \in N^{\downarrow}(v_i) \\ 0, & \text{otherwise.} \end{cases} \quad (10)$$

$$(A_{W_j,s}^{\uparrow})_{i,i'} = \begin{cases} 1, & \text{if } i > i' \text{ and } v_{i'-i} \in N^{\uparrow}(v_i) \\ 0, & \text{otherwise.} \end{cases} \quad (11)$$

In other words, $I_{W_j,s}$ keeps track of whether the current simplex has been visited before within the local window s , while $A_{W_j,s}^{\downarrow}$ and $A_{W_j,s}^{\uparrow}$ keep track of whether the current simplex is lower or upper adjacent to a previously visited simplex within the local window s , respectively.

Random walks

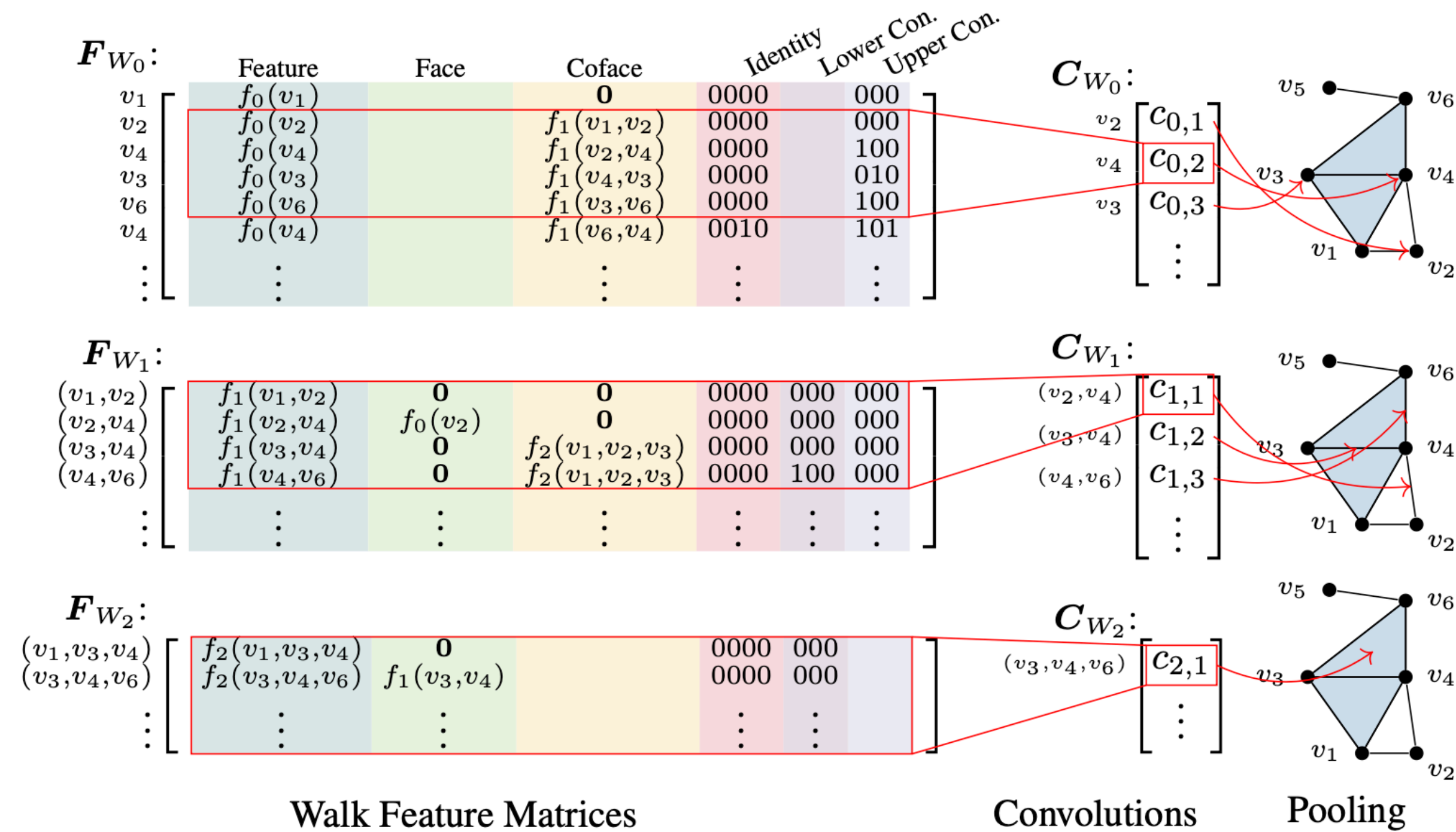


Figure 2: **Information flow in SCRaWI modules for simplex orders 0, 1, and 2.** Using the walks shown in Figure 1, we compute the feature matrices F_0 , F_1 , and F_2 based on simplex features f and a window size of $s = 4$. The feature matrices are convolved with a 1D CNN while keeping track of the central simplex within the window (values to the left of the convolution matrices). Convolved matrices are pooled into simplices grouped by the central simplices.

Table 1: **Imputation accuracies on the Semantic Scholar dataset with varying percentages of missing citations.** We report the mean and standard deviation of the validation accuracies over 10 runs. We compare SCRaWI to the simplicial neural networks SNN, SCNN, SAT, SAN, and MPSN as well as to the representation learning algorithm k -simplex2vec.

Validation Size	Method	Simplex Order					
		0 $n_0 = 352$	1 $n_1 = 1474$	2 $n_2 = 3285$	3 $n_3 = 5019$	4 $n_4 = 5559$	5 $n_5 = 4547$
50%	k -simplex2vec	0.07 ± 0.015	0.07 ± 0.008	0.36 ± 0.016	0.80 ± 0.022	0.91 ± 0.007	0.95 ± 0.003
	SNN	0.54 ± 0.007	0.55 ± 0.005	0.56 ± 0.003	0.57 ± 0.003	0.59 ± 0.004	0.60 ± 0.005
	SCNN	0.54 ± 0.006	0.55 ± 0.004	0.56 ± 0.003	0.58 ± 0.003	0.59 ± 0.003	0.61 ± 0.002
	SAT	0.19 ± 0.000	0.30 ± 0.001	0.22 ± 0.000	0.32 ± 0.001	0.43 ± 0.000	0.48 ± 0.001
	SAN	0.61 ± 0.019	0.79 ± 0.043	0.82 ± 0.008	0.88 ± 0.015	0.92 ± 0.007	0.94 ± 0.011
	SIN (MPSN)	0.94 ± 0.004	0.97 ± 0.006	0.97 ± 0.008	0.95 ± 0.002	0.96 ± 0.004	0.97 ± 0.009
	SCRaWI	0.95 ± 0.029	0.98 ± 0.009	0.96 ± 0.009	0.96 ± 0.032	0.97 ± 0.014	0.96 ± 0.035

SNN and SCNN have been reported by [Yang et al. \(2022a\)](#), SAT and SAN have been reported by [Giusti et al. \(2022\)](#).

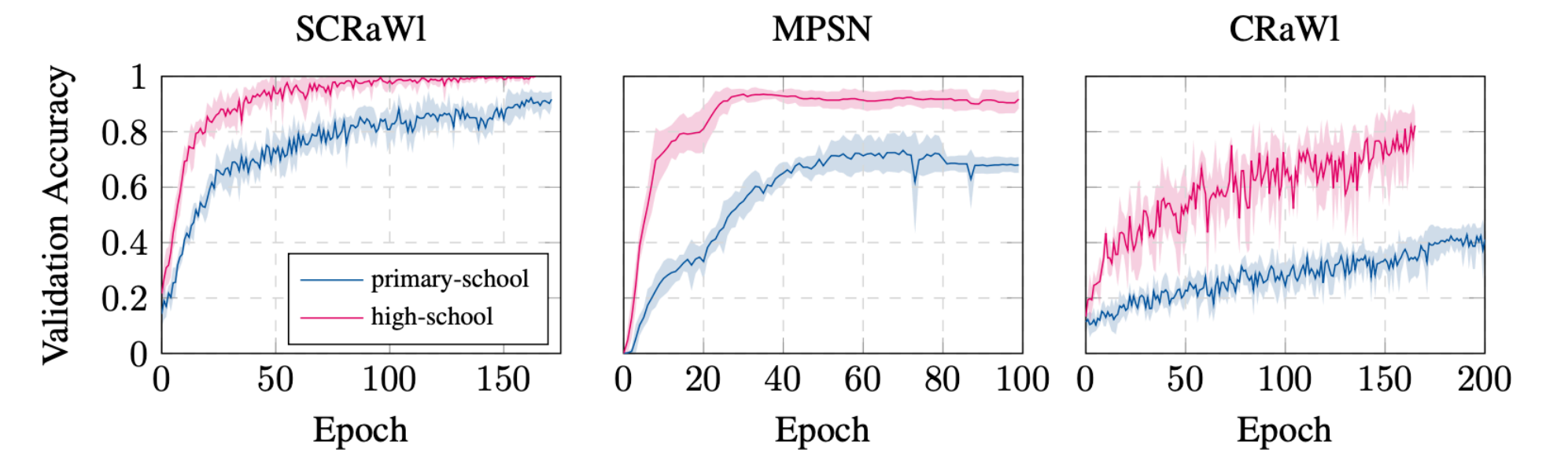


Figure 7: **Vertex classification accuracies of SCRaWI, MPSN, and CRaWI on the social contact datasets.** We report the mean and standard deviation of the validation accuracies over 5 runs.